**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**

Факультет прикладной математики–процессов управления

Программа бакалавриата

**“Большие данные и распределенная цифровая платформа”**

# ОТЧЁТ

По лабораторной работе № 1

По дисциплине “Алгоритмы и структуры данных”

На тему “Исследование задач глобальной оптимизации через генетический и роевой алгоритмы с модификациями.”

Студент гр. 24Б15-пу
Телицин А.Д.

Преподаватель
Дик А.Г.

Санкт-Петербург
2026 г.

# Содержание

**1 Цель работы** **2**

**2 Описание алгоритмов и постановка задачи** **2**

**3 Пошаговое описание работы ГА и РА и их блок-схемы.** **2**
3.1 Генетический алгоритм. . . . . 2
3.2 Роевой алгоритм . . . . . 5

**4 Формализация задачи** **7**

**5 Рекомендации пользователю** **7**

**6 Рекомендации программиста** **7**

**7 Описание контрольного примера.** **8**

**8 Анализ результатов работы алгоритма.** **10**
8.1 Цели анализа . . . . . 10
8.2 Методика проведения экспериментов . . . . . 10
8.3 Исследование генетического алгоритма . . . . . 11
8.3.1 Влияние размера популяции . . . . . 11
8.3.2 Влияние вероятности кроссовера . . . . . 11
8.3.3 Влияние вероятности мутации . . . . . 11
8.3.4 Влияние силы мутации . . . . . 12
8.3.5 Влияние турнирного отбора . . . . . 12
8.3.6 Влияние модификации элитизм . . . . . 13
8.4 Исследование роевого алгоритма . . . . . 13
8.4.1 Влияние размера роя . . . . . 13
8.4.2 Влияние коэффицента инерции . . . . . 14
8.4.3 Влияние когнитивного коэффициента . . . . . 14
8.4.4 Влияние социального коэффициента. . . . . 14
8.4.5 Влияние коэффициента ограничения скорости. . . . . 15
8.5 Сравнение генетического и роевого алгоритма . . . . . 16
8.5.1 Методология сравнения . . . . . 16
8.5.2 Выбор оптимальных параметров для сравнения . . . . . 16
8.5.3 Результаты сравнения . . . . . 16

**9 Выводы** **17**

**10 Листинг кода** **18**

# 1 Цель работы

Целью работы является сравнительный анализ эффективности генетического алгоритма с разными типами кодирования генотипа (вещественный и бинарный) и алгоритма роевого интеллекта в задачах глобальной оптимизации.

# 2 Описание алгоритмов и постановка задачи

**Генетический алгоритм** – это эвристический алгоритм поиска, используемый для решения задач оптимизации и моделирования путём случайного подбора, комбинирования и вариации искомых параметров с использованием механизмов, аналогичных естественному отбору в природе.

Генетический алгоритм является разновидностью эволюционных вычислений, с помощью которых решаются оптимизационные задачи с использованием методов естественной эволюции, таких как наследование, мутации, отбор и кроссинговер.

Отличительной особенностью генетического алгоритма является акцент на использование оператора "скрещивания", который производит операцию рекомбинации решений-кандидатов, роль которых аналогична роли скрещивания в живой природе.

**Роевой алгоритм** (Particle Swarm Optimization – PSO) – это эвристический метод глобальной оптимизации, вдохновлённый социальным поведением живых организмов, таких как стаи птиц, косяки рыб или рои насекомых. Алгоритм моделирует движение группы агентов (частиц) в пространстве поиска, где каждая частица корректирует свою траекторию, ориентируясь на собственный лучший опыт и на опыт всей группы.

Роевой алгоритм является одним из подходов роевого интеллекта и широко применяется для решения многомерных задач оптимизации, в том числе там, где традиционные методы неэффективны.

Отличительной особенностью роевого алгоритма является использование коллективного обмена информацией: каждая частица хранит свою лучшую найденную позицию (личный оптимум) и учитывает глобальную лучшую позицию, обнаруженную всем роем. Благодаря этому обеспечивается баланс между исследованием (exploration) новых областей пространства и эксплуатацией (exploitation) уже найденных перспективных зон. Дополнительно вводится ограничение скорости частиц, предотвращающее чрезмерно большие перемещения и стабилизирующее процесс сходимости.

В работе исследуется работа данных алгоритмов в следующей задаче оптимизации:

$$f(x) = \frac{\sum_{i=1}^{n} \left(x_i^4 - 16x_i^2 + 5x_i\right)}{2} \to \min \tag{1}$$

В работе будет рассмотрен случай $n = 2$ в целях упрощения визуализации движения частиц в процессе работы алгоритма.

# 3 Пошаговое описание работы ГА и РА и их блок-схемы.

## 3.1 Генетический алгоритм.

**Пошаговое описание (версия с вещественным кодированием):**

1. **Инициализация:** Создание начальной популяции из заданного числа особей. Каждая особь представляет собой вектор вещественных чисел, генерируемый

случайным образом в заданном диапазоне.

2. **Оценка приспособленности:** Вычисление значения целевой функции $f(x)$ для каждой особи.

3. **Запись лучшего решения:** Поиск особи с наименьшим значением функции в текущей популяции и сохранение её как лучшего решения, если оно лучше ранее найденного.

4. **Формирование новой популяции.** Пока новая популяция не будет заполнена, выполняются шаги 4.1–4.4:

   (a) **Отбор (турнирный):** Случайным образом выбирается $k$ особей из текущей популяции. Особь с наименьшим значением функции становится первым родителем. Процесс повторяется для выбора второго родителя.

   (b) **Кроссовер (арифметический):** С заданной вероятностью применяется арифметический кроссовер: потомок вычисляется как взвешенная сумма родителей. Если кроссовер не применяется, потомок копируется от первого родителя.

   (c) **Мутация (гауссова):** Для каждой координаты потомка с заданной вероятностью добавляется случайное возмущение из нормального распределения $N(0, \sigma)$. Полученные координаты обрезаются до границ заданного диапазона.

   (d) **Добавление потомка:** Полученная особь добавляется в новую популяцию.

5. **Замена поколения:** Текущая популяция заменяется новой.

6. **Проверка условия останова:** Если достигнуто максимальное число поколений, алгоритм завершается, иначе переход к шагу 2.

**Пошаговое описание (версия с двоичным кодированием):**

1. **Инициализация:** Создание начальной популяции из заданного числа особей. Каждая особь представляет собой закодированный через код Грея вектор вещественных чисел, генерируемый случайным образом в заданном диапазоне.

2. **Оценка приспособленности:** Перевод закодированного кодом Грея числа обратно в вещественный вид. Вычисление значения целевой функции $f(x)$ для каждой особи аналогично варианту с вещественным кодированием.

3. **Запись лучшего решения:** Поиск особи с наименьшим значением функции в текущей популяции и сохранение её как лучшего решения, если оно лучше ранее найденного.

4. **Формирование новой популяции.** Пока новая популяция не будет заполнена, выполняются шаги 4.1–4.4:

   (a) **Отбор (турнирный):** Для каждого родителя случайным образом выбирается заданное число индексов. Из всей популяции берется особь с наименьшим значением целевой функции, выбранная особь становится родителем.

(b) **Кроссовер:** С заданной вероятностью применяется одноточечный кроссовер для каждой координаты: выбирается точка разрыва, слева от которой вставляются гены первого родителя, справа – второго. Если кроссовер не применяется, потомки являются копиями родителей.

(c) **Мутация:** Для каждого бита потомка с заданной вероятностью выполняется инверсия.

(d) **Добавление потомка:** Полученная особь добавляется в новую популяцию.

5. **Замена поколения:** Текущая популяция заменяется новой.

6. **Проверка условия останова:** Если достигнуто максимальное число поколений, алгоритм завершается, иначе переход к шагу 2.

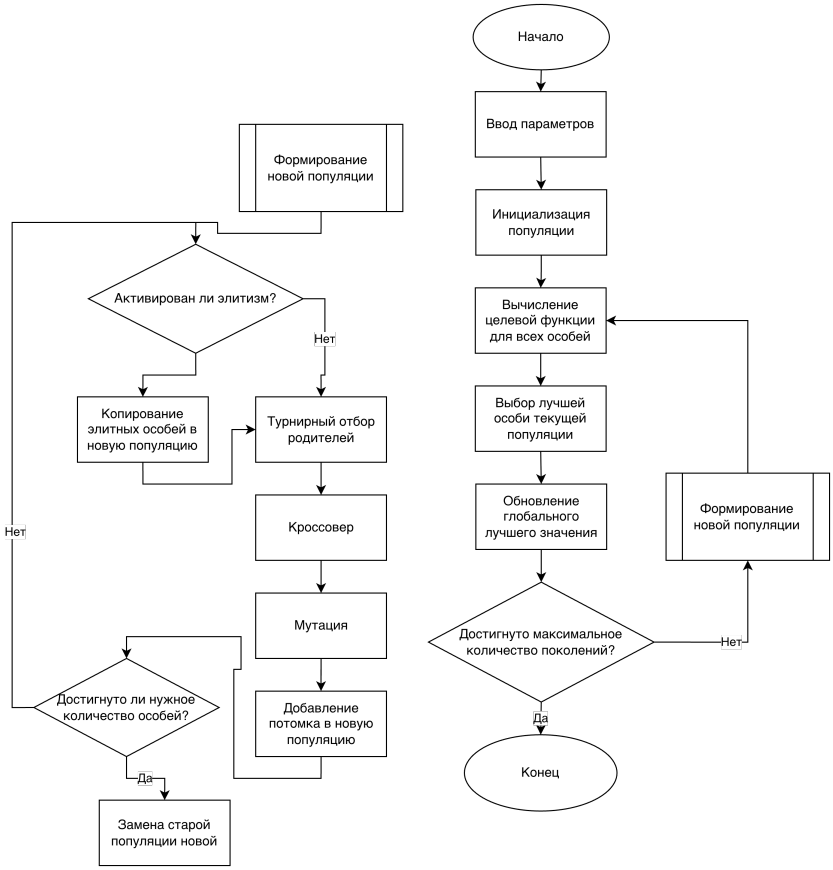


Рис. 1: Блок-схема генетического алгоритма.

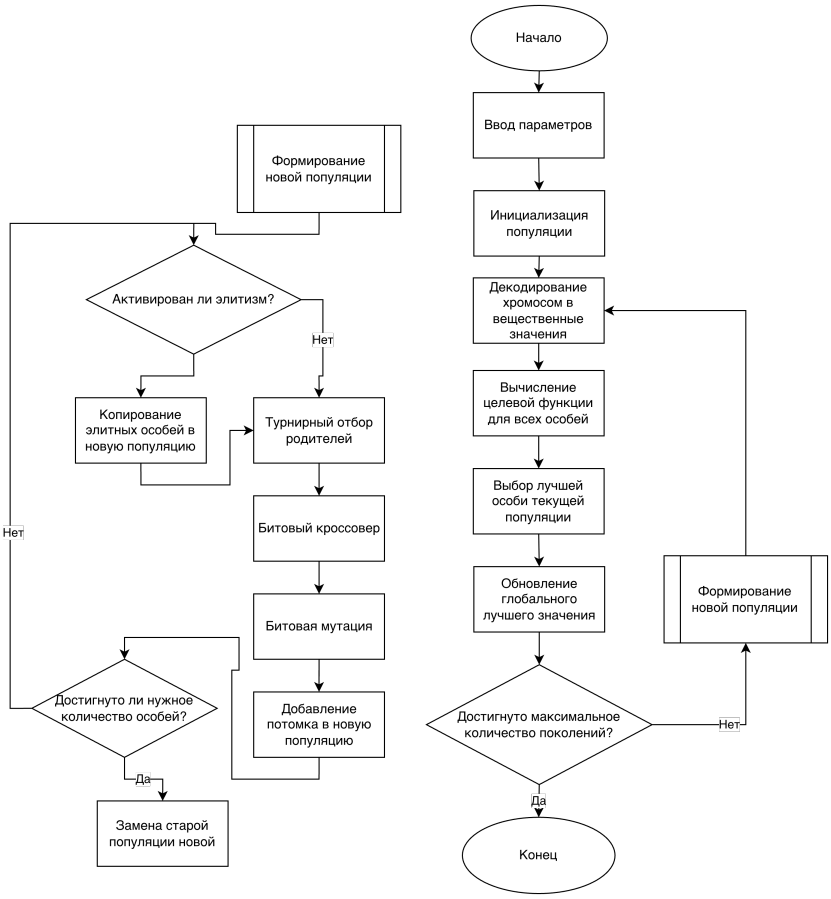


Рис. 2: Блок-схема генетического алгоритма (двоичное кодирование).

### 3.2 Роевой алгоритм

**Пошаговое описание:**

1. **Инициализация роя:** создается заданное число частиц, координаты каждой частицы генерируются случайным образом в заданном диапазоне.

2. **Инициализация скоростей:** для каждой частицы генерируется значение скорости в заданном диапазоне.

3. **Инициализация лучшего локального и глобального значений.**

4. **Сохранение значений в историю.** Сохраняются следующие значения: текущее глобальное лучшее значение целевой функции; точка, задающая это лучшее значение; среднее значение целевой функции; положение всех точек (для визуализации).

5. **Основной цикл, шаги 5.1 – 5.4:**

   (a) **Генерация коэффициентов направления:** Для каждой частицы и каждой координаты генерируются коэффициенты стремления к локальному и глобальному лучшим значениям.

   (b) **Пересчет скорости** по заданной формуле:

   $$v = wv + c_1 r_1 (pbest_x - x) + c_2 r_2 (gbest_x - x)$$

   Опционально может быть применено ограничение скорости, в таком случае скорость обрезается в заданном диапазоне.

(c) **Обновление позиций.**

(d) **Обновление глобального лучшего значения и обновление истории.**

6. **Условие останова:** алгоритм завершается после выполнения заданного числа итераций. Возвращаются следующие значения: лучшая найденная точка, ее значение; история перемещения лучшей точки; история среднего значения функции по рою; история положений всех частиц (для визуализации).

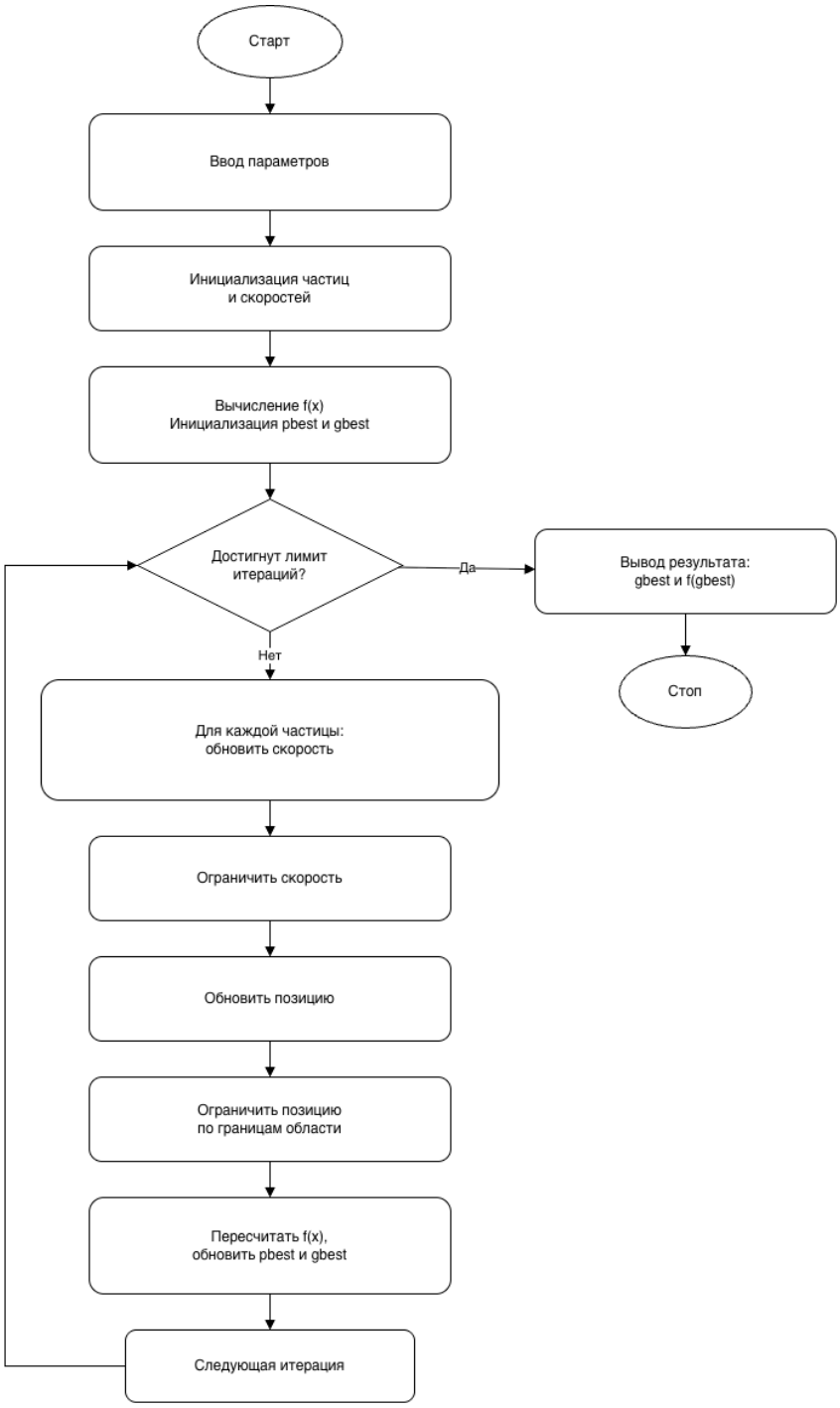


Рис. 3: Блок-схема роевого алгоритма.

## 4 Формализация задачи

Программа реализована на языке Python с использованием следующих библиотек и фреймворков:

1. numpy – используется для более эффективного выполнения арифметических операций и для генерации случайных величин.
2. matplotlib – используется для визуализации работы алгоритмов и для построения графиков.
3. PyQt6 – используется для построения графического интерфейса управления параметрами программы.

Исходный код программы имеет следующую многофайловую архитектуру:

Листинг 1: Файловая архитектура программы.

```
|-- ga.py
|-- gui_qt.py
|-- main.py
|-- pso.py
|-- visualisation.py
`-- requirements.txt
```

## 5 Рекомендации пользователю

Взаимодействие с программой осуществляется через графический интерфейс.

## 6 Рекомендации программиста

**Системные требования:**

- Операционная система Windows/Linux/MacOS
- Свободное место на диске – 10 Мб
- Свободная оперативная память – 200 Мб

**Требования к установленному ПО:**

- Python 3.9+
- numpy, matplotlib, PyQt6. (полный список в файле requirements.txt)

**Инструкция по запуску:**

1. Склонировать репозиторий:

```
git clone https://github.com/pterodactylys/algorithms-and-data
   -structures.git
```

2. Перейти в директорию с программой:

```
cd task_5_ga_and_pso/
```

3. Активировать виртуальное окружение и установить зависимости:

```
python3 -m venv new_env
pip install -r requirements.txt
```

4. Запустить программу:

```
python3 main.py
```

В результате данных действий откроется Qt-окно с интерфейсом программы.

## 7 Описание контрольного примера.

Для проверки корректности работы алгоритма была запущена проверка на задаче оптимизации для функции, описанной выше.

На рисунке ниже изображен интерфейс программы после выполнения алгоритмов. Параметры, с которыми были запущнуты алгоритмы, а также результаты выполнения программы видны на рисунке.

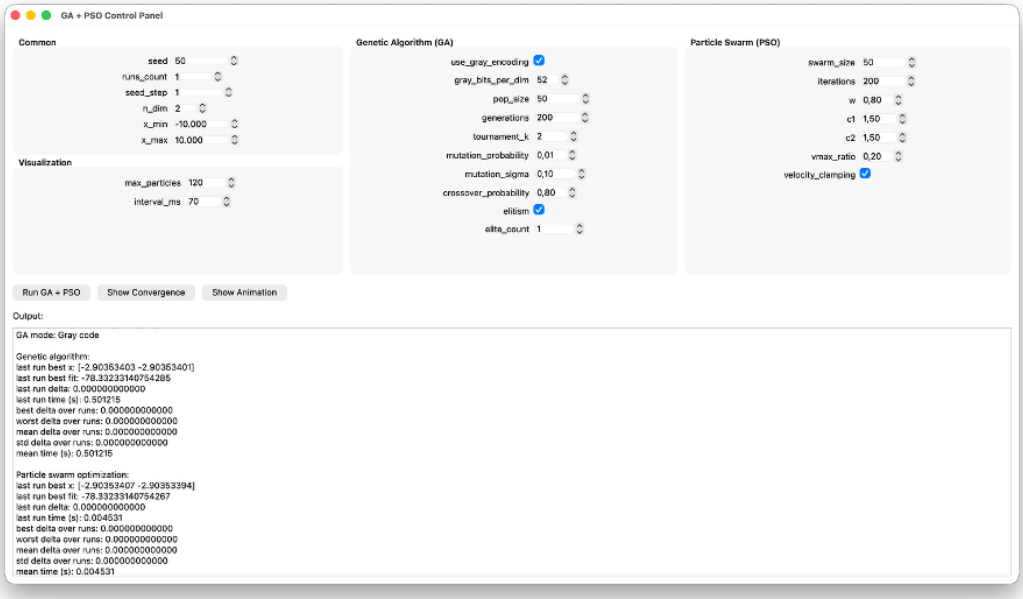


Рис. 4: Интерфейс программы.

В результате анализа вывода программы приходим к следующим выводам:

1. Программа вычисляет значение минимума функции с минимальной погрешностью, разность значений, вычисленных во время работы программы, и значений, известных заранее, является очень маленькой (точность более 12 знаков после запятой). Данная точность достигается как при поиске с помощью ГА, так и с помощью РА. Анализ влияния входных параметров на точность вычисления минимума будет произведен в следующем блоке отчета.

2. Алгоритмы завершаются за короткий промежуток времени, 0.5 секунды для ГА и 0.005 секунды для РА. Анализ влияния входных параметров на скорость выполнения алгоритмов и сравнение алгоритмов будут произведены в следующем блоке отчета.

Кроме вычисления минимума функции, программа позволяет визуализировать полученные результаты: в программу встроена функциональность по построению графика зависимости значения найденного минимального значения на данной итерации с номером итерации; программа позволяет визуализировать движения особей (частиц) в ходе работы алгоритмов.

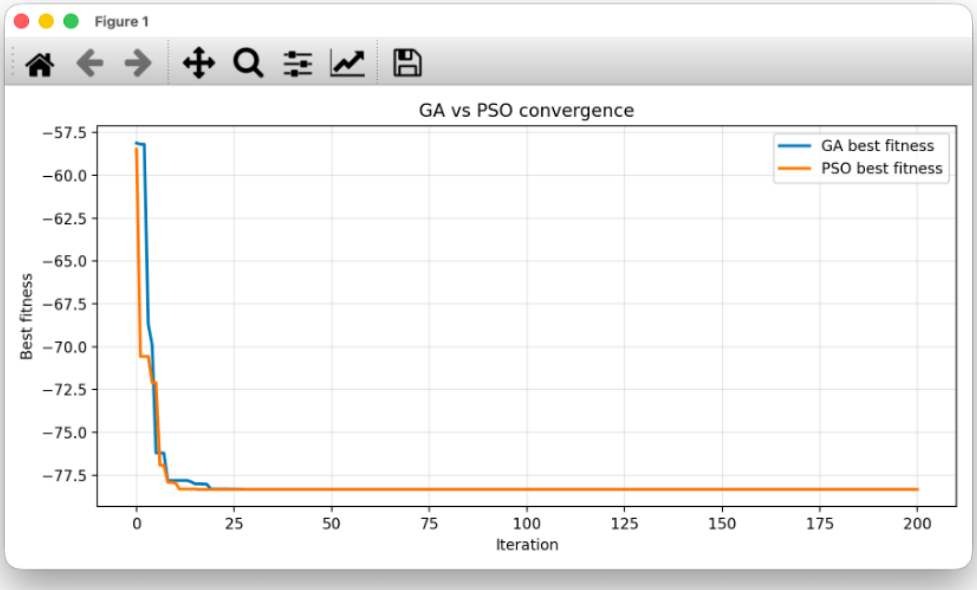


Рис. 5: График зависимости лучшего найденного значения от номера итерации.

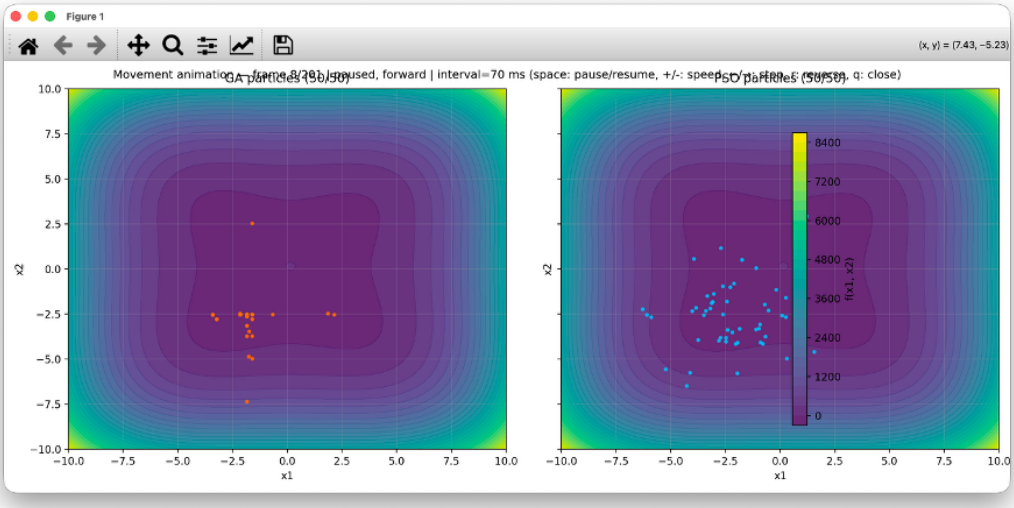


Рис. 6: Кадр из анимации движения частиц.

В результате анализа полученных визуальных представлений работы алгоритма приходим к следующим выводам:

1. Программа позволяет проводить анализ скорости сходимости алгоритмов в зависимости от входных параметров благодаря построению графика изменения лучшего найденного значения.

2. Программа позволяет проводить более точный анализ поведения алгоритма в зависимости от входных параметров благодаря удобному инструменту, позволяющему следить за движением особей (частиц) в ходе работы алгоритмов.

# 8 Анализ результатов работы алгоритма.

## 8.1 Цели анализа

1. Найти оптимальные значения входных параметров для обоих алгоритмов. Оптимальными значениями будем считать те, которые обеспечивают поиск точного значения и при этом не приводят к замедлению работы алгоритма.

2. Провести анализ алгоритмов в стандартном виде с модифицированными версиями и выявить влияние модификации на характеристики работы алгоритма.

3. Выявить качественные и количественные изменения в работе алгоритмов при изменении входных значений.

4. Провести сравнительный анализ работы каждого из алгоритмов, запущенных с оптимальными значениями и определить какой из них является наиболее подходящим для решения задачи оптимизации для имеющейся функции.

## 8.2 Методика проведения экспериментов

Для получения статистически достоверных результатов каждый эксперимент проводился серией из 30 независимых прогонов. Независимость достигается благодаря изменению параметра генерации случайных величин (seed) при каждом новом запуске. Данная мера позволяет оценивать не только скорость и точность работы алгоритма, но и его стабильность.

Для упрощения проведения экспериментов, связанных с многократным запуском программы, был реализован скрипт, который позволяет автоматизировать процесс запуска программы с различными входными параметрами и сохранять полученные результаты в структурированном виде для последущего анализа.

В качестве основных метрик были использованы:

- **Точность** ($\Delta$) – абсолютная ошибка между найденным значением минимума функции и фактическим:

$$\Delta = |f_{\text{true}} - f_{\text{найденное}}|$$

- **Точность по аргументам** ($\Delta_{\text{arg}}$) – Евклидово расстояние между найденной точкой минимума и истинной:

$$d = \sqrt{\sum_{i=1}^{n} (x_i^{\text{true}} - x_i^{\text{найденное}})^2}$$

- **Стабильность** ($\sigma$) – стандартное отклонение ошибки по серии запусков.

- **Скорость сходимости** ($t$) – число итераций, необходимое для достижения ошибки менее $10^{-6}$.

- **Время выполнения** ($\tau$) – среднее время работы алгоритма, секунды.

Исследование проводилось на области $[-10, 10] \times [-10, 10]$. Размерность пространства $n = 2$ выбрана в целях упрощения визуализации.

### 8.3 Исследование генетического алгоритма

#### 8.3.1 Влияние размера популяции

Размер популяции изменялся от 100 до 1900 с шагом 200.

Таблица 1: Исследование влияния размера популяции.

| $N$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 100 | $1.413 \times 10^{0}$ | $4.241 \times 10^{0}$ | $5.651 \times 10^{-1}$ | $7.78 \times 10^{1}$ | $2.077408 \times 10^{-1}$ |
| 300 | $5.925 \times 10^{-8}$ | $4.705 \times 10^{-8}$ | $5.386 \times 10^{-5}$ | $5.90 \times 10^{1}$ | $6.157380 \times 10^{-1}$ |
| 500 | $5.100 \times 10^{-8}$ | $5.860 \times 10^{-8}$ | $4.542 \times 10^{-5}$ | $4.09 \times 10^{1}$ | $1.015055 \times 10^{0}$ |
| 700 | $6.076 \times 10^{-8}$ | $4.5834 \times 10^{-8}$ | $5.450 \times 10^{-5}$ | $3.54 \times 10^{1}$ | $1.422419 \times 10^{0}$ |
| 900 | $4.573 \times 10^{-8}$ | $6.392 \times 10^{-8}$ | $3.836 \times 10^{-5}$ | $3.70 \times 10^{1}$ | $1.826041 \times 10^{0}$ |
| 1100 | $2.169 \times 10^{-8}$ | $2.576 \times 10^{-8}$ | $3.073 \times 10^{-5}$ | $3.66 \times 10^{1}$ | $2.253967 \times 10^{0}$ |
| 1300 | $6.467 \times 10^{-8}$ | $8.1218 \times 10^{-8}$ | $5.3285 \times 10^{-5}$ | $3.68 \times 10^{1}$ | $2.640822 \times 10^{0}$ |
| 1500 | $5.107 \times 10^{-8}$ | $5.5472 \times 10^{-8}$ | $4.662 \times 10^{-5}$ | $3.54 \times 10^{1}$ | $3.135932 \times 10^{0}$ |
| 1700 | $2.246 \times 10^{-8}$ | $1.763 \times 10^{-8}$ | $3.271 \times 10^{-5}$ | $3.07 \times 10^{1}$ | $3.568018 \times 10^{0}$ |
| 1900 | $2.908 \times 10^{-8}$ | $2.790 \times 10^{-8}$ | $3.6402 \times 10^{-5}$ | $2.70 \times 10^{1}$ | $4.023912 \times 10^{0}$ |

С увеличением размера популяции увеличивается и время выполнения, зависимость практически линейная. С ростом количества особей повышается точность и стабильность, но начиная со значения в 300 особей прирост становится практически неразличим.

#### 8.3.2 Влияние вероятности кроссовера

Вероятность кроссовера изменялась от 0.5 до 1 с шагом 0.1.

Таблица 2: Исследование влияния вероятности кроссовера.

| $p$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.500 | $2.948971 \times 10^{-8}$ | $3.772672 \times 10^{-8}$ | $3.396331 \times 10^{-5}$ | $3.330000 \times 10^{1}$ | $1.954774 \times 10^{0}$ |
| 0.600 | $3.372306 \times 10^{-8}$ | $2.822315 \times 10^{-8}$ | $4.017700 \times 10^{-5}$ | $3.380000 \times 10^{1}$ | $2.002842 \times 10^{0}$ |
| 0.700 | $3.872338 \times 10^{-8}$ | $4.584356 \times 10^{-8}$ | $4.059837 \times 10^{-5}$ | $3.290000 \times 10^{1}$ | $2.104592 \times 10^{0}$ |
| 0.800 | $1.324232 \times 10^{-8}$ | $7.873226 \times 10^{-9}$ | $2.639759 \times 10^{-5}$ | $3.890000 \times 10^{1}$ | $2.093505 \times 10^{0}$ |
| 0.900 | $2.826980 \times 10^{-8}$ | $3.049576 \times 10^{-8}$ | $3.319865 \times 10^{-5}$ | $3.510000 \times 10^{1}$ | $2.082939 \times 10^{0}$ |
| 1.000 | $3.706049 \times 10^{-8}$ | $2.524861 \times 10^{-8}$ | $4.373014 \times 10^{-5}$ | $3.600000 \times 10^{1}$ | $2.132884 \times 10^{0}$ |

Наилучшие значения по точности достигаются при $p = 0.8$. В остальных случаях метрики оказались немного хуже. Вероятно, это связано с тем, что при слишком высокой вероятности кроссовера теряется разнообразие в популяции, что может привести к преждевременной сходимости к локальному оптимуму. С другой стороны, при слишком низкой вероятности кроссовера алгоритм может не сохранять достаточное количество информции о лучших решениях, что также может замедлить процесс поиска оптимума.

#### 8.3.3 Влияние вероятности мутации

Вероятность мутации изменялась от 0.05 до 0.5 с шагом 0.05.

Таблица 3: Исследование влияния вероятности мутации.

| $p$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.050 | $4.894176 \times 10^{-10}$ | $5.729121 \times 10^{-10}$ | $4.352485 \times 10^{-6}$ | $3.950000 \times 10^{1}$ | $2.060358 \times 10^{0}$ |
| 0.100 | $1.335820 \times 10^{-13}$ | $1.445321 \times 10^{-13}$ | $6.330867 \times 10^{-8}$ | $3.070000 \times 10^{1}$ | $2.049491 \times 10^{0}$ |
| 0.150 | $1.639933 \times 10^{-11}$ | $1.729413 \times 10^{-11}$ | $8.405745 \times 10^{-7}$ | $2.980000 \times 10^{1}$ | $2.113701 \times 10^{0}$ |
| 0.200 | $3.052935 \times 10^{-9}$ | $4.247264 \times 10^{-9}$ | $1.080024 \times 10^{-5}$ | $2.960000 \times 10^{1}$ | $2.062488 \times 10^{0}$ |
| 0.250 | $1.324232 \times 10^{-8}$ | $7.873226 \times 10^{-9}$ | $2.639759 \times 10^{-5}$ | $3.890000 \times 10^{1}$ | $2.115668 \times 10^{0}$ |
| 0.300 | $1.480270 \times 10^{-7}$ | $1.472215 \times 10^{-7}$ | $7.899599 \times 10^{-5}$ | $6.320000 \times 10^{1}$ | $2.093434 \times 10^{0}$ |
| 0.350 | $5.048306 \times 10^{-7}$ | $3.807409 \times 10^{-7}$ | $1.566273 \times 10^{-4}$ | $4.830000 \times 10^{1}$ | $2.076894 \times 10^{0}$ |
| 0.400 | $9.018358 \times 10^{-7}$ | $8.804333 \times 10^{-7}$ | $2.059078 \times 10^{-4}$ | $1.110000 \times 10^{2}$ | $2.047508 \times 10^{0}$ |
| 0.450 | $1.417778 \times 10^{-6}$ | $7.413373 \times 10^{-7}$ | $2.771483 \times 10^{-4}$ | $7.800000 \times 10^{1}$ | $2.056849 \times 10^{0}$ |
| 0.500 | $2.012143 \times 10^{-6}$ | $2.056267 \times 10^{-6}$ | $2.835861 \times 10^{-4}$ | $7.600000 \times 10^{1}$ | $2.047410 \times 10^{0}$ |

Оптимальным оказалось значение $p = 0.1$. При избыточном увеличении вероятности мутации значительно ухудшается точность поиска оптимума. Помимо этого, растет и количество итераций, необходимых для приближения к окрестности решения.

### 8.3.4 Влияние силы мутации

Коэффициент силы мутации изменялся от 0.1 до 1 с шагом 0.1.

Таблица 4: Исследование влияния силы мутации.

| $\sigma'$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.100 | $1.034680 \times 10^{-8}$ | $1.153397 \times 10^{-8}$ | $2.071156 \times 10^{-5}$ | $3.150000 \times 10^{1}$ | $2.049712 \times 10^{0}$ |
| 0.200 | $1.912376 \times 10^{-8}$ | $1.886803 \times 10^{-8}$ | $2.898458 \times 10^{-5}$ | $3.150000 \times 10^{1}$ | $2.044004 \times 10^{0}$ |
| 0.300 | $7.307934 \times 10^{-8}$ | $1.226486 \times 10^{-7}$ | $5.040931 \times 10^{-5}$ | $3.990000 \times 10^{1}$ | $2.041652 \times 10^{0}$ |
| 0.400 | $1.352250 \times 10^{-7}$ | $1.370886 \times 10^{-7}$ | $7.370266 \times 10^{-5}$ | $4.350000 \times 10^{1}$ | $2.060574 \times 10^{0}$ |
| 0.500 | $1.009254 \times 10^{-7}$ | $1.363793 \times 10^{-7}$ | $6.544238 \times 10^{-5}$ | $3.850000 \times 10^{1}$ | $2.040837 \times 10^{0}$ |
| 0.600 | $1.198760 \times 10^{-7}$ | $9.666718 \times 10^{-8}$ | $7.527097 \times 10^{-5}$ | $5.800000 \times 10^{1}$ | $2.039133 \times 10^{0}$ |
| 0.700 | $2.741389 \times 10^{-7}$ | $1.775513 \times 10^{-7}$ | $1.154844 \times 10^{-4}$ | $7.480000 \times 10^{1}$ | $2.049474 \times 10^{0}$ |
| 0.800 | $5.358516 \times 10^{-7}$ | $6.389252 \times 10^{-7}$ | $1.475894 \times 10^{-4}$ | $5.750000 \times 10^{1}$ | $2.044652 \times 10^{0}$ |
| 0.900 | $5.216789 \times 10^{-7}$ | $4.946148 \times 10^{-7}$ | $1.504138 \times 10^{-4}$ | $8.330000 \times 10^{1}$ | $2.095889 \times 10^{0}$ |
| 1.000 | $5.309839 \times 10^{-7}$ | $4.108561 \times 10^{-7}$ | $1.562627 \times 10^{-4}$ | $8.430000 \times 10^{1}$ | $2.098061 \times 10^{0}$ |

При чрезмерном увеличении данного коэффициента увеличивается количество итераций, необходимых для входа в окрестность глобального минимума. Наиболее оптимальным значением оказалось $\sigma' = 0.1$.

### 8.3.5 Влияние турнирного отбора

Тесты осуществлялись для значений $k = \{2, 3, 5, 7\}$.

Таблица 5: Исследование влияния турнирного отбора.

| $k$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 2 | $1.324232 \times 10^{-8}$ | $7.873226 \times 10^{-9}$ | $2.639759 \times 10^{-5}$ | $3.890000 \times 10^{1}$ | $2.074541 \times 10^{0}$ |
| 3 | $3.552714 \times 10^{-14}$ | $7.105427 \times 10^{-15}$ | $4.218785 \times 10^{-8}$ | $1.650000 \times 10^{1}$ | $2.082557 \times 10^{0}$ |
| 5 | $5.684342 \times 10^{-14}$ | 0 | $3.951726 \times 10^{-8}$ | $1.070000 \times 10^{1}$ | $2.092837 \times 10^{0}$ |
| 7 | $5.684342 \times 10^{-14}$ | 0 | $4.261332 \times 10^{-8}$ | $9.400000 \times 10^{0}$ | $2.061180 \times 10^{0}$ |

С ростом количества особей, участвующих в турнирном отборе, значительно растет точность и скорость поиска минимума. В некоторых случаях данная закономерность может не быть верной, так как алгоритм с высоким значением $k$ может преждевременно сойтись к локальному минимуму.

### 8.3.6 Влияние модификации элитизм

Параметр принимает два значения: Да / Нет.

Таблица 6: Исследование влияния элитизма.

| Элитизм | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| Нет | $9.082491 \times 10^{-6}$ | $1.003952 \times 10^{-5}$ | $5.920785 \times 10^{-4}$ | $5.090000 \times 10^{1}$ | $2.075020 \times 10^{0}$ |
| Да | $1.324232 \times 10^{-8}$ | $7.873226 \times 10^{-9}$ | $2.639759 \times 10^{-5}$ | $3.890000 \times 10^{1}$ | $2.083524 \times 10^{0}$ |

Полученные выводы имеют схожесть с выводами, сделанными в п.8.3.5. В некоторых случаях модификация элитизм может кратно повысить точность поиска и ускорить его, но также могут быть и случаи, когда модификация приведет к преждевременной сходимости к локальному минимуму.

## 8.4 Исследование роевого алгоритма

### 8.4.1 Влияние размера роя

Количество частиц в рое изменялось от 50 до 500 с шагом 50.

Таблица 7: Исследование влияния размера роя.

| $N$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 50 | $3.836931 \times 10^{-13}$ | $5.237621 \times 10^{-13}$ | $1.355096 \times 10^{-7}$ | $8.780000 \times 10^{1}$ | $4.569183 \times 10^{-3}$ |
| 100 | $2.700062 \times 10^{-14}$ | $1.180442 \times 10^{-14}$ | $5.342486 \times 10^{-8}$ | $7.500000 \times 10^{1}$ | $5.205083 \times 10^{-3}$ |
| 150 | $4.547474 \times 10^{-14}$ | $6.122224 \times 10^{-14}$ | $5.423153 \times 10^{-8}$ | $7.030000 \times 10^{1}$ | $5.952908 \times 10^{-3}$ |
| 200 | $2.842171 \times 10^{-14}$ | $1.797547 \times 10^{-14}$ | $4.607717 \times 10^{-8}$ | $6.460000 \times 10^{1}$ | $6.808779 \times 10^{-3}$ |
| 250 | $3.268497 \times 10^{-14}$ | $1.428173 \times 10^{-14}$ | $3.795767 \times 10^{-8}$ | $6.240000 \times 10^{1}$ | $7.133583 \times 10^{-3}$ |
| 300 | $2.842171 \times 10^{-14}$ | $8.987734 \times 10^{-15}$ | $4.810519 \times 10^{-8}$ | $6.170000 \times 10^{1}$ | $8.046446 \times 10^{-3}$ |
| 350 | $3.836931 \times 10^{-14}$ | $1.109903 \times 10^{-14}$ | $4.109045 \times 10^{-8}$ | $5.890000 \times 10^{1}$ | $8.533063 \times 10^{-3}$ |
| 400 | $3.552714 \times 10^{-14}$ | $9.532931 \times 10^{-15}$ | $3.861770 \times 10^{-8}$ | $6.090000 \times 10^{1}$ | $9.173525 \times 10^{-3}$ |
| 450 | $4.405365 \times 10^{-14}$ | $1.180442 \times 10^{-14}$ | $3.741379 \times 10^{-8}$ | $5.800000 \times 10^{1}$ | $1.028253 \times 10^{-2}$ |
| 500 | $3.979039 \times 10^{-14}$ | $1.238874 \times 10^{-14}$ | $4.290907 \times 10^{-8}$ | $5.810000 \times 10^{1}$ | $1.104899 \times 10^{-2}$ |

Значительных изменений при изменении количества частиц в рое не происходит. Вероятно, это происходит из-за особенности текущей конфигурации алгоритма, т.е. качество нахождения результата обеспечивается другими параметрами.

### 8.4.2 Влияние коэффицента инерции

Коэффициент инерции изменялся от 0.4 до 1 с шагом 0.1.

Таблица 8: Исследование влияния коэффициента инерции.

| $w$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.400 | $5.684342 \times 10^{-14}$ | $4.452869 \times 10^{-8}$ | 0 | $2.030000 \times 10^{1}$ | $5.707633 \times 10^{-3}$ |
| 0.500 | $5.684342 \times 10^{-14}$ | $3.974059 \times 10^{-8}$ | 0 | $2.540000 \times 10^{1}$ | $5.647646 \times 10^{-3}$ |
| 0.600 | $5.684342 \times 10^{-14}$ | $3.678255 \times 10^{-8}$ | 0 | $3.150000 \times 10^{1}$ | $5.498088 \times 10^{-3}$ |
| 0.700 | $5.684342 \times 10^{-14}$ | $4.030899 \times 10^{-8}$ | 0 | $4.260000 \times 10^{1}$ | $5.457996 \times 10^{-3}$ |
| 0.800 | $1.861622 \times 10^{-13}$ | $7.641853 \times 10^{-8}$ | $3.891517 \times 10^{-13}$ | $6.740000 \times 10^{1}$ | $5.456429 \times 10^{-3}$ |
| 0.900 | $7.138466 \times 10^{-6}$ | $5.251731 \times 10^{-4}$ | $8.606383 \times 10^{-6}$ | $1.558000 \times 10^{2}$ | $5.320612 \times 10^{-3}$ |
| 1.000 | $4.448560 \times 10^{-3}$ | $1.384260 \times 10^{-2}$ | $4.879384 \times 10^{-3}$ | $1.103000 \times 10^{2}$ | $5.277063 \times 10^{-3}$ |

При избыточном увеличении данного коэффициента наблюдается резкое ухудшение точности поиска минимума функции. Наилучшие результаты достигаются при $w$ в промежутке $[0.4, 0.6]$. Вероятно, что высокие значения данного параметра приводят к тому, что частицы слишком сильно ориентируются на свое предыдущее направление движения, что приводит к снижению чувствительности к информации от других частиц и, как следствие, ухужшению качества поиска оптимума.

### 8.4.3 Влияние когнитивного коэффициента

Когнитивный коэффициент изменялся от 0.5 до 2.5 с шагом 0.25.

Таблица 9: Влияние когнитивного коэффициента.

| $c_1$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.500 | $3.836931 \times 10^{-14}$ | $9.099387 \times 10^{-15}$ | $3.875763 \times 10^{-8}$ | $6.890000 \times 10^{1}$ | $5.513012 \times 10^{-3}$ |
| 0.750 | $5.115908 \times 10^{-14}$ | $6.961869 \times 10^{-15}$ | $4.056561 \times 10^{-8}$ | $6.540000 \times 10^{1}$ | $5.406288 \times 10^{-3}$ |
| 1.000 | $4.263256 \times 10^{-14}$ | $1.271057 \times 10^{-14}$ | $5.018416 \times 10^{-8}$ | $6.480000 \times 10^{1}$ | $5.397892 \times 10^{-3}$ |
| 1.250 | $3.694822 \times 10^{-14}$ | $6.961869 \times 10^{-15}$ | $4.394249 \times 10^{-8}$ | $7.630000 \times 10^{1}$ | $5.402979 \times 10^{-3}$ |
| 1.500 | $1.861622 \times 10^{-13}$ | $3.891517 \times 10^{-13}$ | $7.641853 \times 10^{-8}$ | $6.740000 \times 10^{1}$ | $5.382592 \times 10^{-3}$ |
| 1.750 | $3.930722 \times 10^{-12}$ | $8.171741 \times 10^{-12}$ | $3.318538 \times 10^{-7}$ | $8.350000 \times 10^{1}$ | $5.440967 \times 10^{-3}$ |
| 2.000 | $8.675727 \times 10^{-12}$ | $1.575956 \times 10^{-11}$ | $4.958520 \times 10^{-7}$ | $8.730000 \times 10^{1}$ | $5.360579 \times 10^{-3}$ |
| 2.250 | $2.485440 \times 10^{-9}$ | $2.507115 \times 10^{-9}$ | $1.004312 \times 10^{-5}$ | $1.019000 \times 10^{2}$ | $5.339608 \times 10^{-3}$ |
| 2.500 | $9.477593 \times 10^{-8}$ | $1.145552 \times 10^{-7}$ | $5.945898 \times 10^{-5}$ | $1.363000 \times 10^{2}$ | $5.321762 \times 10^{-3}$ |

Заметно ухудшение точности при увеличении данного коэффициента. Наилучшие результаты достигаются при $c_1$ в промежутке $[0.5, 1.25]$. Вероятно, что высокие значения данного параметра приводят к тому, что частицы слишком сильно ориентируются на свое предыдущее положение, что может привести к сходимости к локальному минимуму.

### 8.4.4 Влияние социального коэффициента.

Социальный коэффициент изменялся от 0.5 до 2.5 с шагом 0.25.

Таблица 10: Влияние социального коэффициента.

| $c_2$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.500 | $5.542233 \times 10^{-14}$ | $4.263256 \times 10^{-15}$ | $3.955374 \times 10^{-8}$ | $5.210000 \times 10^{1}$ | $5.522650 \times 10^{-3}$ |
| 0.750 | $5.115908 \times 10^{-14}$ | $6.961869 \times 10^{-15}$ | $3.834909 \times 10^{-8}$ | $5.640000 \times 10^{1}$ | $5.595587 \times 10^{-3}$ |
| 1.000 | $4.831691 \times 10^{-14}$ | $6.961869 \times 10^{-15}$ | $3.755681 \times 10^{-8}$ | $6.380000 \times 10^{1}$ | $5.442404 \times 10^{-3}$ |
| 1.250 | $4.121148 \times 10^{-14}$ | $7.652779 \times 10^{-15}$ | $3.969039 \times 10^{-8}$ | $6.850000 \times 10^{1}$ | $5.635683 \times 10^{-3}$ |
| 1.500 | $1.861622 \times 10^{-13}$ | $3.891517 \times 10^{-13}$ | $7.641853 \times 10^{-8}$ | $6.740000 \times 10^{1}$ | $5.524584 \times 10^{-3}$ |
| 1.750 | $1.820410 \times 10^{-12}$ | $2.989097 \times 10^{-12}$ | $2.219393 \times 10^{-7}$ | $7.900000 \times 10^{1}$ | $5.386358 \times 10^{-3}$ |
| 2.000 | $4.750262 \times 10^{-11}$ | $5.546970 \times 10^{-11}$ | $1.325882 \times 10^{-6}$ | $9.260000 \times 10^{1}$ | $5.374771 \times 10^{-3}$ |
| 2.250 | $3.235499 \times 10^{-10}$ | $4.022909 \times 10^{-10}$ | $3.425286 \times 10^{-6}$ | $1.118000 \times 10^{2}$ | $5.358404 \times 10^{-3}$ |
| 2.500 | $1.663276 \times 10^{-7}$ | $3.644920 \times 10^{-7}$ | $6.474510 \times 10^{-5}$ | $1.154000 \times 10^{2}$ | $5.333967 \times 10^{-3}$ |

Наилучшие результаты достигаются при $c_2$ в промежутке $[0.5, 1.25]$. Вероятно, что высокие значения данного параметра приводят к тому, что частицы слишком сильно ориентируются на глобальный оптимум, что может привести к сходимости к локальному оптимуму илик тому, что частицы будут искать глобальный оптимум дольше.

### 8.4.5 Влияние коэффициента ограничения скорости.

Коэффициент ограничения скорости изменялся от 0.05 до 0.5 с шагом 0.05.

Таблица 11: Влияние коэффициента ограничения скорости.

| $v_{\max}$ | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| 0.050 | $3.126388 \times 10^{-14}$ | $1.774935 \times 10^{-14}$ | $4.453654 \times 10^{-8}$ | $6.510000 \times 10^{1}$ | $5.492954 \times 10^{-3}$ |
| 0.100 | $2.984279 \times 10^{-14}$ | $9.947598 \times 10^{-15}$ | $4.441813 \times 10^{-8}$ | $6.690000 \times 10^{1}$ | $5.402996 \times 10^{-3}$ |
| 0.150 | $3.552714 \times 10^{-14}$ | $1.711214 \times 10^{-14}$ | $5.078329 \times 10^{-8}$ | $6.050000 \times 10^{1}$ | $5.404604 \times 10^{-3}$ |
| 0.200 | $1.861622 \times 10^{-13}$ | $3.891517 \times 10^{-13}$ | $7.641853 \times 10^{-8}$ | $6.740000 \times 10^{1}$ | $5.411475 \times 10^{-3}$ |
| 0.250 | $2.557954 \times 10^{-14}$ | $1.530556 \times 10^{-14}$ | $4.315051 \times 10^{-8}$ | $7.390000 \times 10^{1}$ | $5.390892 \times 10^{-3}$ |
| 0.300 | $2.984279 \times 10^{-14}$ | $2.242435 \times 10^{-14}$ | $5.226198 \times 10^{-8}$ | $7.610000 \times 10^{1}$ | $5.390400 \times 10^{-3}$ |
| 0.350 | $1.989520 \times 10^{-14}$ | $1.582454 \times 10^{-14}$ | $4.567457 \times 10^{-8}$ | $7.820000 \times 10^{1}$ | $5.388371 \times 10^{-3}$ |
| 0.400 | $3.268497 \times 10^{-14}$ | $3.868382 \times 10^{-14}$ | $5.202514 \times 10^{-8}$ | $8.050000 \times 10^{1}$ | $5.381942 \times 10^{-3}$ |
| 0.450 | $4.405365 \times 10^{-14}$ | $4.981913 \times 10^{-14}$ | $6.636644 \times 10^{-8}$ | $7.710000 \times 10^{1}$ | $5.383692 \times 10^{-3}$ |
| 0.500 | $7.247536 \times 10^{-14}$ | $1.064297 \times 10^{-13}$ | $6.914543 \times 10^{-8}$ | $7.930000 \times 10^{1}$ | $5.385762 \times 10^{-3}$ |

Наилучшие результаты достигаются при $v_{\max}$ в промежутке $[0.2, 0.3]$. При значениях вне этого промежутка наблюдается незначительное ухудшение точности поиска оптимума. Вероятно, что слишком высокие значения данного параметра приводят к слишком быстрому движению частиц, что может привести к тому, что частицы будут перескакивать через оптимум. С другой стороны, слишком низкие значения данного параметра могут сказаться в том, что частицы будут двигаться слишком меделнно, что может привести к тому, что частицы будут дольше искать оптимум, либо к преждевременной сходимости к локальному оптимуму.

Данный результат может быть применен и на случай полного отключения модификации ограничения скорости. При отключении ограничения по скорости полностью, эффект по "перескакиванию"через оптимум будет еще более заметным.

## 8.5 Сравнение генетического и роевого алгоритма

### 8.5.1 Методология сравнения

Качество оптимизации сравнивалось по следующим метрикам:

- $\Delta$ - абсолютная погрешность найденного решения.
- $\Delta_{\text{arg}}$ - абсолютная погрешность аргумента найденного решения.
- $\sigma$ - стандартное отклонение найденного решения.
- $t$ - количество итераций, необходимых для приближения к окрестности решения.
- $\tau$ - время, затраченное на оптимизацию.

Наиболее важными метриками являются $\Delta$ и $\Delta_{\text{arg}}$, так как они напрямую характеризуют качество найденного решения. Метрика $\sigma$ позволяет оценить стабильность алгоритма, а метрики $t$ и $\tau$ позволяют оценить эффективность алгоритма.

Выводы о качестве оптимизации будут производиться на основе сравнения этих метрик для обоих алгоритмов, запущенных с оптимальными параметрами, определенными в предыдущих пунктах.

### 8.5.2 Выбор оптимальных параметров для сравнения

Для сравнения алгоритмов были выбраны следующие параметры:

- Генетический алгоритм (вещественное кодирование): $N = 200$, $k = 3$, $p_{cross} = 0.8$, $p_{mut} = 0.01$, $\sigma' = 0.25$, элитизм включен, $e = 3$
- Генетический алгоритм (бинарное кодирование): $N = 200$, $k = 3$, $p_{cross} = 0.8$, $p_{mut} = 0.25$, $\sigma' = 0.25$, элитизм включен, $e = 3$
- Роевой алгоритм: $N = 120$, $w = 0.4$, $c_1 = 1.25$, $c_2 = 1.20$, $v_{\max} = 0.2$.

### 8.5.3 Результаты сравнения

Таблица 12: Сравнение генетического и роевого алгоритма.

| Алгоритм | $\Delta$ | $\Delta_{\text{arg}}$ | $\sigma$ | $t$ | $\tau$, секунд |
|---|---|---|---|---|---|
| $GA_{Real}$ | $5.613288 \times 10^{-14}$ | $4.066605 \times 10^{-8}$ | $3.097184 \times 10^{-15}$ | $1.65 \times 10^{1}$ | $5.55 \times 10^{-1}$ |
| $GA_{Bin}$ | 0 | $2.096648 \times 10^{-9}$ | 0 | $2.35 \times 10^{1}$ | $5.44 \times 10^{-1}$ |
| $PSO_{VMax}$ | $5.684342 \times 10^{-14}$ | $4.270654 \times 10^{-8}$ | $3.34321 \times 10^{-14}$ | $1.85 \times 10^{1}$ | $1.91 \times 10^{-3}$ |

Оба алгоритма показали высокую точность поиска оптимума. Погрешность $\Delta$ и $\Delta_{\text{arg}}$ оба алгоритма достигли порядка $10^{-14}$ и $10^{-8}$ соответственно, а бинарная версия ГА достигла точности, превышающей точность представления вещественных чисел, что свидетельствует об эффективности обоих алгоритмов в решении данной задачи оптимизации.

Несмотря на схожую точность, роевой алгоритм оказался значительно быстрее генетического алгоритма, что указывает на его более высокую эффективность

в данной задаче. Вероятно, что подход, используемый в роевом алгоритме является более удачным в данном случае.

Однако, в ходе экспериментов было замечено, что генетический алгоритм оказался более чувсвительным к небольшим изменениям параметров, что в некоторых случаях приводило к значительному ухудшению качества оптимизации. С другой стороны, роевой алгоритм показал намного более стабильные результаты при изменении параметров, что указывает на его более высокую устойчивость к выбору параметров и стабильность в поиске решения.

## 9 Выводы

В ходе данной лабораторной работы были исследованы и протестированы два алгоритма в задаче оптимизации: генетический алгоритм и роевой алгоритм. Были проведены эксперименты по исследованию влияния различных параметров на качество оптимизации, а также проведено сравнение эффективности этих алгоритмов. Были получены следующие выводы:

- Оба алгоритма показали высокую точность в поиске оптимума, что свидетельствует об их применимости для решения задач оптимизации.
- Бинарная версия генетического алгоритма показала наилучшую точность среди рассмотренных методов.
- Роевой алгоритм показал значительно более высокую эффективность по сравнению с генетическим алгоритмом при прочих равных.
- Генетический алгоритм оказался менее стабильным в поиске решения и более чувствительным к выбору параметров, что может затрудить подбор этих параметров.
- Роевой алгоритм показал более стабильные результаты при изменении параметров.
- Выбор оптимальных параметров - наиболее важный этап при решении задач с использованием этих алгоритмов.
- В случае необходимости быстрого выполнения задачи наиболее предпочтительным алгоритмом для данной задачи является роевой алгоритм, так как он показал более высокую эффективность и стабильность в поиске решения.
- При необходимости достижения максимально возможной точности оптимизации, наиболее предпочтительным алгоритмом является бинарная версия генетического алгоритма, так как она показала наилучшую точность среди рассмотренных методов.

Полученные результаты могут распространяться в полном объеме только на оптимизацию выбранной функции, так как оптимальные параметры алгоритмов могут варьироваться в зависимости от конкретной задачи. Также, стоит отметить, что результаты могут меняться в зависимости от начальной инициализации алгоритмов, так как оба алгоритма являются стохастическими. Тем не менее, проведенное сравнение может показать общие тенденции в поведении алгоритмов и их чувствительность к различным параметрам.

## 10 Листинг кода

В данном разделе приведены исходные коды основных модулей программы.

**Листинг 2. Исходный код файла** `main.py`

```
import argparse
import time

from ga import GAConfig, objective, run_ga
from metrics import argument_error, format_scientific, iterations_to_convergence, summarize
from pso import PSOConfig, run_pso
from visualisation import animate_particles_movement, plot_ga_pso_convergence

import numpy as np


def run_cli(seed: int = 50, runs: int = 1, seed_step: int = 1):
    cfg = GAConfig(
        n_dim=2,
        x_min=-10.0,
        x_max=10.0,
        pop_size=1000,
        generations=200,
        tournament_k=2,
        mutation_probability=0.25,
        mutation_sigma=0.25,
        crossover_probability=0.8,
        elitism=True,
        elite_count=1,
    )
    ga_times = []
    pso_times = []
    ga_fit_errors = []
    pso_fit_errors = []
    ga_arg_errors = []
    pso_arg_errors = []
    ga_conv_iters = []
    pso_conv_iters = []

    pso_cfg = PSOConfig(
        n_dim=cfg.n_dim,
        x_min=cfg.x_min,
        x_max=cfg.x_max,
        swarm_size=120,
        iterations=200,
        w=0.8,
        c1=1.5,
        c2=1.5,
        vmax_ratio=0.2,
        velocity_clamping=True,
    )
```

```
    ga_res = None
    pso_res = None

    x_reference = np.full(cfg.n_dim, -2.903534)
    f_reference = float(objective(x_reference))

    for run_idx in range(runs):
        run_seed = seed + run_idx * seed_step
        random_generator = np.random.default_rng(run_seed)

        ga_t0 = time.perf_counter()
        ga_res = run_ga(cfg, random_generator)
        ga_time_s = time.perf_counter() - ga_t0

        pso_t0 = time.perf_counter()
        pso_res = run_pso(pso_cfg, random_generator)
        pso_time_s = time.perf_counter() - pso_t0

        ga_times.append(float(ga_time_s))
        pso_times.append(float(pso_time_s))

        ga_fit_errors.append(abs(float(ga_res["best_fit"]) - f_reference))
        pso_fit_errors.append(abs(float(pso_res["best_fit"]) - f_reference))
        ga_arg_errors.append(argument_error(ga_res["best_x"], x_reference))
        pso_arg_errors.append(argument_error(pso_res["best_x"], x_reference))
        ga_conv_iters.append(iterations_to_convergence(ga_res["best_fit_history"], tolerance=
        pso_conv_iters.append(iterations_to_convergence(pso_res["best_fit_history"], tolerano

    assert ga_res is not None and pso_res is not None

    ga_fit_stats = summarize(ga_fit_errors)
    pso_fit_stats = summarize(pso_fit_errors)
    ga_arg_stats = summarize(ga_arg_errors)
    pso_arg_stats = summarize(pso_arg_errors)
    ga_conv_stats = summarize(ga_conv_iters)
    pso_conv_stats = summarize(pso_conv_iters)
    ga_time_stats = summarize(ga_times)
    pso_time_stats = summarize(pso_times)

    print(f"Runs: {runs} | seed_start={seed} | seed_step={seed_step}")
    print()
    print("Genetic algorithm: ")
    print("last run best x:", ga_res["best_x"])
    print("last run best fit:", ga_res["best_fit"])
    print("accuracy (|f_best - f_ref|):")
    print("  best:", format_scientific(ga_fit_stats["best"]))
    print("  worst:", format_scientific(ga_fit_stats["worst"]))
    print("  mean:", format_scientific(ga_fit_stats["mean"]))
    print("  std:", format_scientific(ga_fit_stats["std"]))
    print("argument accuracy (||x_best - x_ref||):")
    print("  best:", format_scientific(ga_arg_stats["best"]))
    print("  worst:", format_scientific(ga_arg_stats["worst"]))
    print("  mean:", format_scientific(ga_arg_stats["mean"]))
```

```
    print("  std:", format_scientific(ga_arg_stats["std"]))
    print("stability (std of |f_best - f_ref|):", format_scientific(ga_fit_stats["std"]))
    print("iterations to convergence:")
    print("  best:", int(round(ga_conv_stats["best"])))
    print("  worst:", int(round(ga_conv_stats["worst"])))
    print("  mean:", format_scientific(ga_conv_stats["mean"]))
    print("  std:", format_scientific(ga_conv_stats["std"]))
    print("execution time (s):")
    print("  best:", format_scientific(ga_time_stats["best"]))
    print("  worst:", format_scientific(ga_time_stats["worst"]))
    print("  mean:", format_scientific(ga_time_stats["mean"]))
    print("  std:", format_scientific(ga_time_stats["std"]))

    print()
    print("Particle swarm optimization:")
    print("last run best x:", pso_res["best_x"])
    print("last run best fit:", pso_res["best_fit"])
    print("accuracy (|f_best - f_ref|):")
    print("  best:", format_scientific(pso_fit_stats["best"]))
    print("  worst:", format_scientific(pso_fit_stats["worst"]))
    print("  mean:", format_scientific(pso_fit_stats["mean"]))
    print("  std:", format_scientific(pso_fit_stats["std"]))
    print("argument accuracy (||x_best - x_ref||):")
    print("  best:", format_scientific(pso_arg_stats["best"]))
    print("  worst:", format_scientific(pso_arg_stats["worst"]))
    print("  mean:", format_scientific(pso_arg_stats["mean"]))
    print("  std:", format_scientific(pso_arg_stats["std"]))
    print("stability (std of |f_best - f_ref|):", format_scientific(pso_fit_stats["std"]))
    print("iterations to convergence:")
    print("  best:", int(round(pso_conv_stats["best"])))
    print("  worst:", int(round(pso_conv_stats["worst"])))
    print("  mean:", format_scientific(pso_conv_stats["mean"]))
    print("  std:", format_scientific(pso_conv_stats["std"]))
    print("execution time (s):")
    print("  best:", format_scientific(pso_time_stats["best"]))
    print("  worst:", format_scientific(pso_time_stats["worst"]))
    print("  mean:", format_scientific(pso_time_stats["mean"]))
    print("  std:", format_scientific(pso_time_stats["std"]))

    if runs > 1:
        print()
        print("Per-run summary:")
        for run_idx in range(runs):
            run_seed = seed + run_idx * seed_step
            print(
                f"run {run_idx + 1}, seed={run_seed}: "
                f"GA acc={format_scientific(ga_fit_errors[run_idx])}, "
                f"GA arg={format_scientific(ga_arg_errors[run_idx])}, "
                f"GA conv={ga_conv_iters[run_idx]}, GA t={format_scientific(ga_times[run_idx]
                f"PSO acc={format_scientific(pso_fit_errors[run_idx])}, "
                f"PSO arg={format_scientific(pso_arg_errors[run_idx])}, "
                f"PSO conv={pso_conv_iters[run_idx]}, PSO t={format_scientific(pso_times[run_
            )
```

```
    plot_ga_pso_convergence(ga_res, pso_res)
    animate_particles_movement(ga_res, pso_res, x_min=cfg.x_min, x_max=cfg.x_max, max_particl


def main():
    parser = argparse.ArgumentParser(description="GA + PSO runner")
    parser.add_argument("--cli", action="store_true", help="Run in CLI mode (without Qt GUI)'
    parser.add_argument("--seed", type=int, default=50, help="Start seed")
    parser.add_argument("--runs", type=int, default=1, help="Number of consecutive runs")
    parser.add_argument("--seed-step", type=int, default=1, help="Seed increment between runs
    args = parser.parse_args()

    if args.cli:
        run_cli(seed=args.seed, runs=max(1, args.runs), seed_step=max(1, args.seed_step))
        return

    try:
        from gui_qt import launch_gui
    except Exception as exc:
        print("Qt GUI is unavailable:", exc)
        print("Starting CLI mode. Use --cli to force CLI explicitly.")
        run_cli(seed=args.seed, runs=max(1, args.runs), seed_step=max(1, args.seed_step))
        return

    launch_gui()

if __name__ == "__main__":
    main()
```

**Листинг 3. Исходный код файла** ga.py

```
from dataclasses import dataclass

import numpy as np


@dataclass
class GAConfig:
    n_dim: int = 2
    x_min: float = -10.0
    x_max: float = 10.0

    pop_size: int = 1000
    generations: int = 200
    tournament_k: int = 2
    mutation_probability: float = 0.25
    mutation_sigma: float = 0.25
    crossover_probability: float = 0.8

    elitism: bool = True
    elite_count: int = 1
```

```
    # Gray-code mode params
    gray_bits_per_dim: int = 20


def objective(x: np.ndarray) -> np.ndarray:
    arr = np.asarray(x, dtype=float)
    return 0.5 * np.sum(arr**4 - 16.0 * arr**2 + 5.0 * arr, axis=-1)


def selection(pop: np.ndarray, fit: np.ndarray, k: int, random_generator: np.random.Generator
    idx = random_generator.integers(0, len(pop), size=k)
    best = idx[np.argmin(fit[idx])]
    return pop[best].copy()


def arithmetic_crossover(p1: np.ndarray, p2: np.ndarray, random_gen: np.random.Generator) ->
    alpha = random_gen.uniform()
    c1 = p1 * alpha + p2 * (1 - alpha)
    c2 = p2 * alpha + p1 * (1 - alpha)
    return c1, c2


def mutation(
    child: np.ndarray,
    mutation_prob: float,
    sigma: float,
    x_min: float,
    x_max: float,
    rng: np.random.Generator,
) -> np.ndarray:
    mask = rng.random(child.shape) < mutation_prob
    noise = rng.normal(0.0, sigma, size=child.shape)
    child = child + mask * noise
    return np.clip(child, x_min, x_max)


def _int_to_gray(value: int) -> int:
    return value ^ (value >> 1)


def _gray_to_int(gray: int) -> int:
    value = gray
    shift = 1
    while (gray >> shift) > 0:
        value ^= gray >> shift
        shift += 1
    return value


def _int_to_bits(value: int, bits_per_dim: int) -> np.ndarray:
    bits = np.zeros(bits_per_dim, dtype=np.uint8)
    for bit in range(bits_per_dim):
        shift = bits_per_dim - 1 - bit
```

```
        bits[bit] = (value >> shift) & 1
    return bits


def _bits_to_int(bits: np.ndarray) -> int:
    value = 0
    for bit in bits:
        value = (value << 1) | int(bit)
    return value


def _encode_real_vector_to_gray_bits(x: np.ndarray, x_min: float, x_max: float, bits_per_dim:
    x = np.asarray(x, dtype=float)
    levels = (1 << bits_per_dim) - 1
    if levels <= 0:
        raise ValueError("gray_bits_per_dim must be >= 1")

    scaled = np.clip((x - x_min) / (x_max - x_min), 0.0, 1.0)
    ints = np.rint(scaled * levels).astype(int)

    encoded = np.zeros((x.shape[0], bits_per_dim), dtype=np.uint8)
    for dim in range(x.shape[0]):
        gray = _int_to_gray(int(ints[dim]))
        encoded[dim] = _int_to_bits(gray, bits_per_dim)
    return encoded


def _decode_gray_bits_to_real_vector(bits: np.ndarray, x_min: float, x_max: float, bits_per_d
    bits = np.asarray(bits, dtype=np.uint8)
    levels = (1 << bits_per_dim) - 1
    decoded = np.zeros(bits.shape[0], dtype=float)

    for dim in range(bits.shape[0]):
        gray = _bits_to_int(bits[dim])
        value_int = _gray_to_int(gray)
        decoded[dim] = x_min + (value_int / levels) * (x_max - x_min)
    return decoded


def _decode_population_gray(pop_bits: np.ndarray, x_min: float, x_max: float, bits_per_dim: i
    pop_size, n_dim, _ = pop_bits.shape
    pop_real = np.zeros((pop_size, n_dim), dtype=float)
    for i in range(pop_size):
        pop_real[i] = _decode_gray_bits_to_real_vector(pop_bits[i], x_min, x_max, bits_per_di
    return pop_real


def _selection_index(fit: np.ndarray, k: int, random_generator: np.random.Generator) -> int:
    idx = random_generator.integers(0, len(fit), size=k)
    return int(idx[np.argmin(fit[idx])])


def _gray_crossover(
```

```
    p1_bits: np.ndarray,
    p2_bits: np.ndarray,
    crossover_probability: float,
    random_generator: np.random.Generator,
) -> tuple[np.ndarray, np.ndarray]:
    c1 = p1_bits.copy()
    c2 = p2_bits.copy()

    if random_generator.random() >= crossover_probability:
        return c1, c2

    n_dim, bits_per_dim = p1_bits.shape
    for dim in range(n_dim):
        if bits_per_dim < 2:
            continue
        point = int(random_generator.integers(1, bits_per_dim))
        c1[dim, point:] = p2_bits[dim, point:]
        c2[dim, point:] = p1_bits[dim, point:]
    return c1, c2


def _gray_mutation(child_bits: np.ndarray, mutation_probability: float, random_generator: np.
    mask = random_generator.random(child_bits.shape) < mutation_probability
    child_bits[mask] = 1 - child_bits[mask]
    return child_bits


def run_ga_gray(cfg: GAConfig, random_generator: np.random.Generator) -> dict:
    bits_per_dim = int(cfg.gray_bits_per_dim)
    if bits_per_dim < 1:
        raise ValueError("gray_bits_per_dim must be >= 1")

    # Инициализация реальной популяции и кодирование в Gray bits
    pop_real = random_generator.uniform(cfg.x_min, cfg.x_max, size=(cfg.pop_size, cfg.n_dim))
    pop_bits = np.zeros((cfg.pop_size, cfg.n_dim, bits_per_dim), dtype=np.uint8)
    for i in range(cfg.pop_size):
        pop_bits[i] = _encode_real_vector_to_gray_bits(pop_real[i], cfg.x_min, cfg.x_max, bit

    best_fit_history = []
    mean_fit_history = []
    best_x_history = []
    population_history = [pop_real.copy()]

    for _ in range(cfg.generations):
        pop_real = _decode_population_gray(pop_bits, cfg.x_min, cfg.x_max, bits_per_dim)
        fit = objective(pop_real)

        best_idx = int(np.argmin(fit))
        best_fit = float(fit[best_idx])
        best_x = pop_real[best_idx].copy()

        best_fit_history.append(best_fit)
        mean_fit_history.append(float(np.mean(fit)))
```

```
        best_x_history.append(best_x)

        new_pop_bits = []
        if cfg.elitism:
            elite_count = int(np.clip(cfg.elite_count, 0, cfg.pop_size))
            if elite_count > 0:
                elite_indices = np.argsort(fit)[:elite_count]
                for idx in elite_indices:
                    new_pop_bits.append(pop_bits[idx].copy())

        while len(new_pop_bits) < cfg.pop_size:
            p1_idx = _selection_index(fit, cfg.tournament_k, random_generator)
            p2_idx = _selection_index(fit, cfg.tournament_k, random_generator)

            c1_bits, c2_bits = _gray_crossover(
                pop_bits[p1_idx],
                pop_bits[p2_idx],
                cfg.crossover_probability,
                random_generator,
            )

            c1_bits = _gray_mutation(c1_bits, cfg.mutation_probability, random_generator)
            c2_bits = _gray_mutation(c2_bits, cfg.mutation_probability, random_generator)

            new_pop_bits.append(c1_bits)
            if len(new_pop_bits) < cfg.pop_size:
                new_pop_bits.append(c2_bits)

        pop_bits = np.asarray(new_pop_bits, dtype=np.uint8)
        pop_real = _decode_population_gray(pop_bits, cfg.x_min, cfg.x_max, bits_per_dim)
        population_history.append(pop_real.copy())

    final_real = _decode_population_gray(pop_bits, cfg.x_min, cfg.x_max, bits_per_dim)
    final_fit = objective(final_real)
    best_idx = int(np.argmin(final_fit))

    return {
        "best_x": final_real[best_idx].copy(),
        "best_fit": float(final_fit[best_idx]),
        "best_fit_history": np.asarray(best_fit_history),
        "mean_fit_history": np.asarray(mean_fit_history),
        "best_x_history": np.asarray(best_x_history),
        "population_history": np.asarray(population_history),
    }


def run_ga(cfg: GAConfig, random_generator: np.random.Generator) -> dict:
    pop = random_generator.uniform(cfg.x_min, cfg.x_max, size=(cfg.pop_size, cfg.n_dim))

    best_fit_history = []
    mean_fit_history = []
    best_x_history = []
    population_history = [pop.copy()]
```

```
    for _ in range(cfg.generations):
        fit = objective(pop)

        best_idx = np.argmin(fit)
        elite = pop[best_idx].copy()
        elite_fit = fit[best_idx]

        best_fit_history.append(elite_fit)
        mean_fit_history.append(np.mean(fit))
        best_x_history.append(elite.copy())

        new_pop = []
        if cfg.elitism:
            elite_count = int(np.clip(cfg.elite_count, 0, cfg.pop_size))
            if elite_count > 0:
                elite_indices = np.argsort(fit)[:elite_count]
                elites = pop[elite_indices].copy()
                new_pop.extend(elites)

        while len(new_pop) < cfg.pop_size:
            p1 = selection(pop, fit, cfg.tournament_k, random_generator)
            p2 = selection(pop, fit, cfg.tournament_k, random_generator)

            if random_generator.random() < cfg.crossover_probability:
                c1, c2 = arithmetic_crossover(p1, p2, random_generator)
            else:
                c1, c2 = p1.copy(), p2.copy()

            c1 = mutation(c1, cfg.mutation_probability, cfg.mutation_sigma, cfg.x_min, cfg.x_
            c2 = mutation(c2, cfg.mutation_probability, cfg.mutation_sigma, cfg.x_min, cfg.x_

            new_pop.append(c1)
            if len(new_pop) < cfg.pop_size:
                new_pop.append(c2)

        pop = np.array(new_pop, dtype=float)
        population_history.append(pop.copy())

    final_fit = objective(pop)
    best_idx = np.argmin(final_fit)

    return {
        "best_x": pop[best_idx].copy(),
        "best_fit": float(final_fit[best_idx]),
        "best_fit_history": np.array(best_fit_history),
        "mean_fit_history": np.array(mean_fit_history),
        "best_x_history": np.array(best_x_history),
        "population_history": np.array(population_history),
    }


def demo_gray_encoding_example() -> None:
```

```
    """
    Демонстрация кодирования вещественных чисел в Gray code:
    real -> scaled int -> gray int -> bits -> decode back to real.
    """
    x_min = -10.0
    x_max = 10.0
    bits_per_dim = 52

    print("=== Quantization step by bit depth ===")
    for bits in (8, 16, 24, 32, 40, 48, 52):
        levels_i = (1 << bits) - 1
        step = (x_max - x_min) / levels_i
        print(f"bits={bits:2d} -> step={step:.3e}")
    print()

    # Пример для одной координаты
    x_value = np.array([2.75], dtype=float)
    levels = (1 << bits_per_dim) - 1
    scaled = np.clip((x_value - x_min) / (x_max - x_min), 0.0, 1.0)
    int_value = int(np.rint(scaled[0] * levels))
    gray_value = _int_to_gray(int_value)
    gray_bits = _int_to_bits(gray_value, bits_per_dim)
    decoded_real = _decode_gray_bits_to_real_vector(gray_bits.reshape(1, -1), x_min, x_max, b

    print("=== Gray encoding demo (single value) ===")
    print(f"range: [{x_min}, {x_max}], bits={bits_per_dim}, levels={levels}")
    print(f"x = {x_value[0]:.6f}")
    print(f"scaled int = {int_value}")
    print(f"gray int = {gray_value}")
    print("gray bits =", "".join(str(int(b)) for b in gray_bits))
    print(f"decoded x = {decoded_real:.6f}")
    print(f"abs error = {abs(decoded_real - x_value[0]):.6f}")

    # Проверка свойства соседних кодов (Gray соседние числа отличаются на 1 бит)
    print("\n=== Neighbor property demo ===")
    base = int_value
    nxt = min(base + 1, levels)
    g1 = _int_to_bits(_int_to_gray(base), bits_per_dim)
    g2 = _int_to_bits(_int_to_gray(nxt), bits_per_dim)
    hamming = int(np.sum(g1 != g2))
    print(f"int {base} -> gray {''.join(str(int(b)) for b in g1)}")
    print(f"int {nxt} -> gray {''.join(str(int(b)) for b in g2)}")
    print(f"hamming distance = {hamming}")

    # Пример для вектора координат
    x_vec = np.array([-2.903534, 1.125, 7.75], dtype=float)
    encoded_vec = _encode_real_vector_to_gray_bits(x_vec, x_min, x_max, bits_per_dim)
    decoded_vec = _decode_gray_bits_to_real_vector(encoded_vec, x_min, x_max, bits_per_dim)
    print("\n=== Vector demo ===")
    for i in range(x_vec.shape[0]):
        bits_str = "".join(str(int(b)) for b in encoded_vec[i])
        err = abs(decoded_vec[i] - x_vec[i])
        print(f"x[{i}]={x_vec[i]: .6f} -> bits={bits_str} -> decoded={decoded_vec[i]: .6f} |
```

```
if __name__ == "__main__":
    demo_gray_encoding_example()
```

**Листинг 4. Исходный код файла** `pso.py`

```
from dataclasses import dataclass
from ga import objective

import numpy as np


@dataclass
class PSOConfig:
    n_dim: int = 2
    x_min: float = -10.0
    x_max: float = 10.0

    swarm_size: int = 100
    iterations: int = 200
    w: float = 0.7
    c1: float = 1.5
    c2: float = 1.5
    vmax_ratio: float = 0.2
    velocity_clamping: bool = True


def run_pso(cfg: PSOConfig, random_generator: np.random.Generator):
    swarm_size = cfg.swarm_size
    n_dim = cfg.n_dim

    x = random_generator.uniform(cfg.x_min, cfg.x_max, size=(swarm_size, n_dim))
    vmax = cfg.vmax_ratio * (cfg.x_max - cfg.x_min)
    v = random_generator.uniform(-vmax, vmax, size=(swarm_size, n_dim))

    pbest_x = x.copy()
    pbest_fit = objective(pbest_x)

    gbest_index = int(np.argmin(pbest_fit))
    gbest_x = pbest_x[gbest_index].copy()
    gbest_fit = float(pbest_fit[gbest_index])

    best_fit_history = [gbest_fit]
    best_x_history = [gbest_x.copy()]
    mean_fit_history = [float(np.mean(pbest_fit))]
    swarm_history = [x.copy()]

    for q in range(cfg.iterations):
        r1 = random_generator.random(size=(swarm_size, n_dim))
        r2 = random_generator.random(size=(swarm_size, n_dim))

        v = (
```

```
            cfg.w * v
            + cfg.c1 * r1 * (pbest_x - x)
            + cfg.c2 * r2 * (gbest_x - x)
        )

        if cfg.velocity_clamping:
            v = np.clip(v, -vmax, vmax)

        x = x + v
        x = np.clip(x, cfg.x_min, cfg.x_max)

        current_fit = objective(x)

        improved = current_fit < pbest_fit
        pbest_x[improved] = x[improved]
        pbest_fit[improved] = current_fit[improved]

        gbest_index = int(np.argmin(pbest_fit))
        if float(pbest_fit[gbest_index]) < gbest_fit:
            gbest_fit = float(pbest_fit[gbest_index])
            gbest_x = pbest_x[gbest_index].copy()

        best_fit_history.append(gbest_fit)
        best_x_history.append(gbest_x.copy())
        mean_fit_history.append(float(np.mean(current_fit)))
        swarm_history.append(x.copy())

    return {
        "best_x": gbest_x,
        "best_fit": gbest_fit,
        "best_fit_history": np.asarray(best_fit_history),
        "mean_fit_history": np.asarray(mean_fit_history),
        "best_x_history": np.asarray(best_x_history),
        "swarm_history": np.asarray(swarm_history),
    }
```

**Листинг 5. Исходный код файла** gui_qt.py

```
from __future__ import annotations

import time
import numpy as np

from ga import GAConfig, objective, run_ga, run_ga_gray
from metrics import argument_error, format_scientific, iterations_to_convergence, summarize
from pso import PSOConfig, run_pso
from visualisation import animate_particles_movement, plot_ga_pso_convergence


def launch_gui() -> None:
    try:
        from PySide6.QtCore import Qt
        from PySide6.QtWidgets import (
```

```
        QApplication,
        QCheckBox,
        QDoubleSpinBox,
        QFormLayout,
        QGroupBox,
        QHBoxLayout,
        QLabel,
        QMainWindow,
        QMessageBox,
        QPushButton,
        QSizePolicy,
        QSpinBox,
        QTextEdit,
        QVBoxLayout,
        QWidget,
    )
except Exception as exc:  # pragma: no cover
    raise RuntimeError(
        "Qt GUI requires PySide6. Install it with: pip install PySide6"
    ) from exc

class MainWindow(QMainWindow):
    def __init__(self) -> None:
        super().__init__()
        self.setWindowTitle("GA + PSO Control Panel")
        self.resize(1320, 860)
        self.setMinimumSize(1180, 740)
        self.setStyleSheet("QGroupBox { font-weight: 600; } QGroupBox::title { font-size:

        self.ga_res: dict | None = None
        self.pso_res: dict | None = None
        self.ga_cfg: GAConfig | None = None
        self.pso_cfg: PSOConfig | None = None

        central = QWidget(self)
        self.setCentralWidget(central)
        root = QVBoxLayout(central)
        root.setContentsMargins(12, 12, 12, 12)
        root.setSpacing(10)

        common_box = self._common_group()
        ga_box = self._ga_group()
        pso_box = self._pso_group()
        viz_box = self._viz_group()
        ga_box.setMinimumWidth(470)
        pso_box.setMinimumWidth(470)
        self._configure_group_box(common_box)
        self._configure_group_box(ga_box)
        self._configure_group_box(pso_box)
        self._configure_group_box(viz_box)

        top_row = QHBoxLayout()
        top_row.setSpacing(10)
```

```
        left_col = QVBoxLayout()
        left_col.setSpacing(10)
        left_col.addWidget(common_box, stretch=1)
        left_col.addWidget(viz_box, stretch=1)

        right_col = QHBoxLayout()
        right_col.setSpacing(10)
        right_col.addWidget(ga_box, stretch=1)
        right_col.addWidget(pso_box, stretch=1)

        top_row.addLayout(left_col, stretch=2)
        top_row.addLayout(right_col, stretch=4)

        root.addLayout(top_row, stretch=2)

        root.addLayout(self._actions_row())

        self.output = QTextEdit(self)
        self.output.setReadOnly(True)
        self.output.setPlaceholderText("Run algorithms to see output...")
        self.output.setMinimumHeight(220)
        root.addWidget(QLabel("Output:"))
        root.addWidget(self.output, stretch=3)

    def _configure_group_box(self, box: QGroupBox) -> None:
        box.setMinimumHeight(180)
        box.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)

    def _common_group(self) -> QGroupBox:
        box = QGroupBox("Common")
        form = QFormLayout(box)

        self.seed = QSpinBox()
        self.seed.setRange(0, 10_000_000)
        self.seed.setValue(50)

        self.runs_count = QSpinBox()
        self.runs_count.setRange(1, 10_000)
        self.runs_count.setValue(1)

        self.seed_step = QSpinBox()
        self.seed_step.setRange(1, 1_000_000)
        self.seed_step.setValue(1)

        self.n_dim = QSpinBox()
        self.n_dim.setRange(1, 50)
        self.n_dim.setValue(2)

        self.x_min = QDoubleSpinBox()
        self.x_min.setDecimals(3)
        self.x_min.setRange(-1_000.0, 1_000.0)
        self.x_min.setValue(-10.0)
```

```
        self.x_max = QDoubleSpinBox()
        self.x_max.setDecimals(3)
        self.x_max.setRange(-1_000.0, 1_000.0)
        self.x_max.setValue(10.0)

        form.addRow("seed", self.seed)
        form.addRow("runs_count", self.runs_count)
        form.addRow("seed_step", self.seed_step)
        form.addRow("n_dim", self.n_dim)
        form.addRow("x_min", self.x_min)
        form.addRow("x_max", self.x_max)
        return box

    def _ga_group(self) -> QGroupBox:
        box = QGroupBox("Genetic Algorithm (GA)")
        form = QFormLayout(box)

        self.ga_use_gray = QCheckBox()
        self.ga_use_gray.setChecked(False)

        self.ga_gray_bits_per_dim = QSpinBox()
        self.ga_gray_bits_per_dim.setRange(1, 60)
        self.ga_gray_bits_per_dim.setValue(52)

        self.ga_pop_size = QSpinBox()
        self.ga_pop_size.setRange(2, 200_000)
        self.ga_pop_size.setValue(1000)

        self.ga_generations = QSpinBox()
        self.ga_generations.setRange(1, 100_000)
        self.ga_generations.setValue(200)

        self.ga_tournament_k = QSpinBox()
        self.ga_tournament_k.setRange(1, 1000)
        self.ga_tournament_k.setValue(2)

        self.ga_mutation_probability = QDoubleSpinBox()
        self.ga_mutation_probability.setRange(0.0, 1.0)
        self.ga_mutation_probability.setSingleStep(0.01)
        self.ga_mutation_probability.setValue(0.25)

        self.ga_mutation_sigma = QDoubleSpinBox()
        self.ga_mutation_sigma.setRange(0.0, 100.0)
        self.ga_mutation_sigma.setSingleStep(0.01)
        self.ga_mutation_sigma.setValue(0.25)

        self.ga_crossover_probability = QDoubleSpinBox()
        self.ga_crossover_probability.setRange(0.0, 1.0)
        self.ga_crossover_probability.setSingleStep(0.01)
        self.ga_crossover_probability.setValue(0.8)

        self.ga_elitism = QCheckBox()
```

```
        self.ga_elitism.setChecked(True)

        self.ga_elite_count = QSpinBox()
        self.ga_elite_count.setRange(0, 10_000)
        self.ga_elite_count.setValue(1)

        form.addRow("use_gray_encoding", self.ga_use_gray)
        form.addRow("gray_bits_per_dim", self.ga_gray_bits_per_dim)
        form.addRow("pop_size", self.ga_pop_size)
        form.addRow("generations", self.ga_generations)
        form.addRow("tournament_k", self.ga_tournament_k)
        form.addRow("mutation_probability", self.ga_mutation_probability)
        form.addRow("mutation_sigma", self.ga_mutation_sigma)
        form.addRow("crossover_probability", self.ga_crossover_probability)
        form.addRow("elitism", self.ga_elitism)
        form.addRow("elite_count", self.ga_elite_count)
        return box

    def _pso_group(self) -> QGroupBox:
        box = QGroupBox("Particle Swarm (PSO)")
        form = QFormLayout(box)

        self.pso_swarm_size = QSpinBox()
        self.pso_swarm_size.setRange(2, 200_000)
        self.pso_swarm_size.setValue(120)

        self.pso_iterations = QSpinBox()
        self.pso_iterations.setRange(1, 100_000)
        self.pso_iterations.setValue(200)

        self.pso_w = QDoubleSpinBox()
        self.pso_w.setRange(0.0, 5.0)
        self.pso_w.setSingleStep(0.01)
        self.pso_w.setValue(0.8)

        self.pso_c1 = QDoubleSpinBox()
        self.pso_c1.setRange(0.0, 10.0)
        self.pso_c1.setSingleStep(0.01)
        self.pso_c1.setValue(1.5)

        self.pso_c2 = QDoubleSpinBox()
        self.pso_c2.setRange(0.0, 10.0)
        self.pso_c2.setSingleStep(0.01)
        self.pso_c2.setValue(1.5)

        self.pso_vmax_ratio = QDoubleSpinBox()
        self.pso_vmax_ratio.setRange(0.0, 5.0)
        self.pso_vmax_ratio.setSingleStep(0.01)
        self.pso_vmax_ratio.setValue(0.2)

        self.pso_velocity_clamping = QCheckBox()
        self.pso_velocity_clamping.setChecked(True)
```

```
        form.addRow("swarm_size", self.pso_swarm_size)
        form.addRow("iterations", self.pso_iterations)
        form.addRow("w", self.pso_w)
        form.addRow("c1", self.pso_c1)
        form.addRow("c2", self.pso_c2)
        form.addRow("vmax_ratio", self.pso_vmax_ratio)
        form.addRow("velocity_clamping", self.pso_velocity_clamping)
        return box

    def _viz_group(self) -> QGroupBox:
        box = QGroupBox("Visualization")
        form = QFormLayout(box)

        self.viz_max_particles = QSpinBox()
        self.viz_max_particles.setRange(1, 10_000)
        self.viz_max_particles.setValue(120)

        self.viz_interval_ms = QSpinBox()
        self.viz_interval_ms.setRange(10, 5_000)
        self.viz_interval_ms.setValue(70)

        form.addRow("max_particles", self.viz_max_particles)
        form.addRow("interval_ms", self.viz_interval_ms)
        return box

    def _actions_row(self) -> QHBoxLayout:
        row = QHBoxLayout()

        run_btn = QPushButton("Run GA + PSO")
        run_btn.clicked.connect(self.run_algorithms)

        conv_btn = QPushButton("Show Convergence")
        conv_btn.clicked.connect(self.show_convergence)

        anim_btn = QPushButton("Show Animation")
        anim_btn.clicked.connect(self.show_animation)

        row.addWidget(run_btn)
        row.addWidget(conv_btn)
        row.addWidget(anim_btn)
        row.addStretch(1)
        return row

    def _read_configs(self) -> tuple[GAConfig, PSOConfig, int, int, int]:
        x_min = float(self.x_min.value())
        x_max = float(self.x_max.value())
        if x_min >= x_max:
            raise ValueError("x_min must be less than x_max")

        n_dim = int(self.n_dim.value())
        seed = int(self.seed.value())
        runs_count = int(self.runs_count.value())
        seed_step = int(self.seed_step.value())
```

```
        ga_cfg = GAConfig(
            n_dim=n_dim,
            x_min=x_min,
            x_max=x_max,
            pop_size=int(self.ga_pop_size.value()),
            generations=int(self.ga_generations.value()),
            tournament_k=int(self.ga_tournament_k.value()),
            mutation_probability=float(self.ga_mutation_probability.value()),
            mutation_sigma=float(self.ga_mutation_sigma.value()),
            crossover_probability=float(self.ga_crossover_probability.value()),
            elitism=bool(self.ga_elitism.isChecked()),
            elite_count=int(self.ga_elite_count.value()),
            gray_bits_per_dim=int(self.ga_gray_bits_per_dim.value()),
        )

        pso_cfg = PSOConfig(
            n_dim=n_dim,
            x_min=x_min,
            x_max=x_max,
            swarm_size=int(self.pso_swarm_size.value()),
            iterations=int(self.pso_iterations.value()),
            w=float(self.pso_w.value()),
            c1=float(self.pso_c1.value()),
            c2=float(self.pso_c2.value()),
            vmax_ratio=float(self.pso_vmax_ratio.value()),
            velocity_clamping=bool(self.pso_velocity_clamping.isChecked()),
        )
        return ga_cfg, pso_cfg, seed, runs_count, seed_step

    def run_algorithms(self) -> None:
        try:
            ga_cfg, pso_cfg, seed, runs_count, seed_step = self._read_configs()
            QApplication.setOverrideCursor(Qt.WaitCursor)

            ga_times = []
            pso_times = []
            ga_fit_errors = []
            pso_fit_errors = []
            ga_arg_errors = []
            pso_arg_errors = []
            ga_conv_iters = []
            pso_conv_iters = []

            x_reference = np.full(ga_cfg.n_dim, -2.903534)
            f_reference = float(objective(x_reference))

            ga_runner = run_ga_gray if self.ga_use_gray.isChecked() else run_ga
            ga_mode = "Gray code" if self.ga_use_gray.isChecked() else "Real-coded"

            ga_res = None
            pso_res = None
```

```
        for run_idx in range(runs_count):
            run_seed = seed + run_idx * seed_step
            rng = np.random.default_rng(run_seed)

            ga_t0 = time.perf_counter()
            ga_res = ga_runner(ga_cfg, rng)
            ga_time_s = time.perf_counter() - ga_t0

            pso_t0 = time.perf_counter()
            pso_res = run_pso(pso_cfg, rng)
            pso_time_s = time.perf_counter() - pso_t0

            ga_times.append(float(ga_time_s))
            pso_times.append(float(pso_time_s))

            ga_fit_errors.append(abs(float(ga_res["best_fit"]) - f_reference))
            pso_fit_errors.append(abs(float(pso_res["best_fit"]) - f_reference))
            ga_arg_errors.append(argument_error(ga_res["best_x"], x_reference))
            pso_arg_errors.append(argument_error(pso_res["best_x"], x_reference))
            ga_conv_iters.append(iterations_to_convergence(ga_res["best_fit_history"]
            pso_conv_iters.append(iterations_to_convergence(pso_res["best_fit_history

        assert ga_res is not None and pso_res is not None
        self.ga_cfg = ga_cfg
        self.pso_cfg = pso_cfg
        self.ga_res = ga_res
        self.pso_res = pso_res

        ga_fit_stats = summarize(ga_fit_errors)
        pso_fit_stats = summarize(pso_fit_errors)
        ga_arg_stats = summarize(ga_arg_errors)
        pso_arg_stats = summarize(pso_arg_errors)
        ga_conv_stats = summarize(ga_conv_iters)
        pso_conv_stats = summarize(pso_conv_iters)
        ga_time_stats = summarize(ga_times)
        pso_time_stats = summarize(pso_times)

        text = []
        text.append(f"Runs: {runs_count} | seed_start={seed} | seed_step={seed_step}'
        text.append(f"GA mode: {ga_mode}")
        text.append("")

        text.append("Genetic algorithm:")
        text.append(f"last run best x: {ga_res['best_x']}")
        text.append(f"last run best fit: {ga_res['best_fit']}")
        text.append("accuracy (|f_best - f_ref|):")
        text.append(f"  best: {format_scientific(ga_fit_stats['best'])}")
        text.append(f"  worst: {format_scientific(ga_fit_stats['worst'])}")
        text.append(f"  mean: {format_scientific(ga_fit_stats['mean'])}")
        text.append(f"  std: {format_scientific(ga_fit_stats['std'])}")
        text.append("argument accuracy (||x_best - x_ref||):")
        text.append(f"  best: {format_scientific(ga_arg_stats['best'])}")
        text.append(f"  worst: {format_scientific(ga_arg_stats['worst'])}")
```

```
        text.append(f"  mean: {format_scientific(ga_arg_stats['mean'])}")
        text.append(f"  std: {format_scientific(ga_arg_stats['std'])}")
        text.append(f"stability (std of |f_best - f_ref|): {format_scientific(ga_fit_
        text.append("iterations to convergence:")
        text.append(f"  best: {int(round(ga_conv_stats['best']))}")
        text.append(f"  worst: {int(round(ga_conv_stats['worst']))}")
        text.append(f"  mean: {format_scientific(ga_conv_stats['mean'])}")
        text.append(f"  std: {format_scientific(ga_conv_stats['std'])}")
        text.append("execution time (s):")
        text.append(f"  best: {format_scientific(ga_time_stats['best'])}")
        text.append(f"  worst: {format_scientific(ga_time_stats['worst'])}")
        text.append(f"  mean: {format_scientific(ga_time_stats['mean'])}")
        text.append(f"  std: {format_scientific(ga_time_stats['std'])}")
        text.append("")

        text.append("Particle swarm optimization:")
        text.append(f"last run best x: {pso_res['best_x']}")
        text.append(f"last run best fit: {pso_res['best_fit']}")
        text.append("accuracy (|f_best - f_ref|):")
        text.append(f"  best: {format_scientific(pso_fit_stats['best'])}")
        text.append(f"  worst: {format_scientific(pso_fit_stats['worst'])}")
        text.append(f"  mean: {format_scientific(pso_fit_stats['mean'])}")
        text.append(f"  std: {format_scientific(pso_fit_stats['std'])}")
        text.append("argument accuracy (||x_best - x_ref||):")
        text.append(f"  best: {format_scientific(pso_arg_stats['best'])}")
        text.append(f"  worst: {format_scientific(pso_arg_stats['worst'])}")
        text.append(f"  mean: {format_scientific(pso_arg_stats['mean'])}")
        text.append(f"  std: {format_scientific(pso_arg_stats['std'])}")
        text.append(f"stability (std of |f_best - f_ref|): {format_scientific(pso_fit
        text.append("iterations to convergence:")
        text.append(f"  best: {int(round(pso_conv_stats['best']))}")
        text.append(f"  worst: {int(round(pso_conv_stats['worst']))}")
        text.append(f"  mean: {format_scientific(pso_conv_stats['mean'])}")
        text.append(f"  std: {format_scientific(pso_conv_stats['std'])}")
        text.append("execution time (s):")
        text.append(f"  best: {format_scientific(pso_time_stats['best'])}")
        text.append(f"  worst: {format_scientific(pso_time_stats['worst'])}")
        text.append(f"  mean: {format_scientific(pso_time_stats['mean'])}")
        text.append(f"  std: {format_scientific(pso_time_stats['std'])}")

        if runs_count > 1:
            text.append("")
            text.append("Per-run summary:")
            for run_idx in range(runs_count):
                run_seed = seed + run_idx * seed_step
                text.append(
                    f"run {run_idx + 1}, seed={run_seed}: "
                    f"GA acc={format_scientific(ga_fit_errors[run_idx])}, "
                    f"GA arg={format_scientific(ga_arg_errors[run_idx])}, "
                    f"GA conv={ga_conv_iters[run_idx]}, GA t={format_scientific(ga_ti
                    f"PSO acc={format_scientific(pso_fit_errors[run_idx])}, "
                    f"PSO arg={format_scientific(pso_arg_errors[run_idx])}, "
                    f"PSO conv={pso_conv_iters[run_idx]}, PSO t={format_scientific(ps
```

```
                )

            self.output.setPlainText("\n".join(text))

        except Exception as exc:
            QMessageBox.critical(self, "Error", str(exc))
        finally:
            QApplication.restoreOverrideCursor()

    def _ensure_results(self) -> bool:
        if self.ga_res is None or self.pso_res is None:
            QMessageBox.information(self, "Info", "Run algorithms first.")
            return False
        return True

    def show_convergence(self) -> None:
        if not self._ensure_results():
            return
        plot_ga_pso_convergence(self.ga_res, self.pso_res)

    def show_animation(self) -> None:
        if not self._ensure_results():
            return
        assert self.ga_cfg is not None
        if self.ga_cfg.n_dim != 2:
            QMessageBox.warning(self, "Unsupported", "Animation works only for n_dim = 2.
            return
        animate_particles_movement(
            self.ga_res,
            self.pso_res,
            x_min=self.ga_cfg.x_min,
            x_max=self.ga_cfg.x_max,
            max_particles=int(self.viz_max_particles.value()),
            interval_ms=int(self.viz_interval_ms.value()),
        )

app = QApplication.instance() or QApplication([])
window = MainWindow()
window.show()
app.exec()
```

**Листинг 6. Исходный код файла** `visualisation.py`

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FuncAnimation


_ANIMATION_HOLD = []


def plot_ga_convergence(ga_result: dict) -> None:
    history = np.asarray(ga_result["best_fit_history"], dtype=float)
```

```
    plt.figure(figsize=(8, 4))
    plt.plot(history, label="GA best fitness", linewidth=2)
    plt.xlabel("Generation")
    plt.ylabel("Best fitness")
    plt.title("Genetic Algorithm convergence")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()


def plot_ga_pso_convergence(ga_result: dict, pso_result: dict) -> None:
    ga_history = np.asarray(ga_result["best_fit_history"], dtype=float)
    pso_history = np.asarray(pso_result["best_fit_history"], dtype=float)

    plt.figure(figsize=(9, 4.5))
    plt.plot(ga_history, label="GA best fitness", linewidth=2)
    plt.plot(pso_history, label="PSO best fitness", linewidth=2)
    plt.xlabel("Iteration")
    plt.ylabel("Best fitness")
    plt.title("GA vs PSO convergence")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()


def _objective_2d_grid(x_min: float, x_max: float, y_min: float, y_max: float, points: int =
    xs = np.linspace(x_min, x_max, points)
    ys = np.linspace(y_min, y_max, points)
    xg, yg = np.meshgrid(xs, ys)
    zg = 0.5 * ((xg**4 - 16 * xg**2 + 5 * xg) + (yg**4 - 16 * yg**2 + 5 * yg))
    return xg, yg, zg


def animate_particles_movement(
    ga_result: dict,
    pso_result: dict,
    x_min: float,
    x_max: float,
    max_particles: int = 120,
    interval_ms: int = 60,
) -> FuncAnimation:
    ga_history = np.asarray(ga_result.get("population_history"))
    pso_history = np.asarray(pso_result.get("swarm_history"))

    if ga_history.ndim != 3 or ga_history.shape[-1] != 2:
        raise ValueError("GA animation works only for n_dim=2 and requires population_history
    if pso_history.ndim != 3 or pso_history.shape[-1] != 2:
        raise ValueError("PSO animation works only for n_dim=2 and requires swarm_history")

    xg, yg, zg = _objective_2d_grid(x_min, x_max, x_min, x_max, points=200)
```

```
fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.8), sharex=True, sharey=True)
for ax in axes:
    contour = ax.contourf(xg, yg, zg, levels=30, cmap="viridis", alpha=0.85)
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(x_min, x_max)
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.grid(alpha=0.2)
fig.colorbar(contour, ax=axes.ravel().tolist(), shrink=0.88, label="f(x1, x2)")

ga_particles = ga_history.shape[1]
pso_particles = pso_history.shape[1]
ga_count = min(max_particles, ga_particles)
pso_count = min(max_particles, pso_particles)

ga_idx = np.linspace(0, ga_particles - 1, ga_count, dtype=int)
pso_idx = np.linspace(0, pso_particles - 1, pso_count, dtype=int)

ga_scatter = axes[0].scatter([], [], s=14, c="#f97316", alpha=0.9, edgecolors="none")
pso_scatter = axes[1].scatter([], [], s=14, c="#38bdf8", alpha=0.9, edgecolors="none")

axes[0].set_title(f"GA particles ({ga_count}/{ga_particles})")
axes[1].set_title(f"PSO particles ({pso_count}/{pso_particles})")

total_frames = int(max(ga_history.shape[0], pso_history.shape[0]))
state = {
    "paused": False,
    "interval_ms": max(10, int(interval_ms)),
    "frame": 0,
    "direction": 1,
}

def _frame_to_index(frame: int, steps: int) -> int:
    if total_frames <= 1 or steps <= 1:
        return 0
    return int(round(frame * (steps - 1) / (total_frames - 1)))

def _title(frame: int) -> str:
    status = "paused" if state["paused"] else "running"
    direction = "forward" if state["direction"] > 0 else "backward"
    return (
        f"Movement animation - frame {frame + 1}/{total_frames} | "
        f"{status}, {direction} | interval={state['interval_ms']} ms "
        f"(space: pause/resume, +/-: speed, ←/→: step, r: reverse, q: close)"
    )

def _draw_current_frame():
    frame = state["frame"]
    ga_step = _frame_to_index(frame, ga_history.shape[0])
    pso_step = _frame_to_index(frame, pso_history.shape[0])

    ga_points = ga_history[ga_step, ga_idx, :]
    pso_points = pso_history[pso_step, pso_idx, :]
```

```
    ga_scatter.set_offsets(ga_points)
    pso_scatter.set_offsets(pso_points)

    fig.suptitle(_title(frame), fontsize=11)
    return ga_scatter, pso_scatter

def update(_):
    artists = _draw_current_frame()
    if not state["paused"]:
        state["frame"] = (state["frame"] + state["direction"]) % total_frames
    return artists

def on_key(event):
    key = (event.key or "").lower()
    if key == " ":
        if state["paused"]:
            animation.event_source.start()
        else:
            animation.event_source.stop()
        state["paused"] = not state["paused"]
        fig.canvas.draw_idle()
    elif key in {"+", "="}:
        state["interval_ms"] = max(10, state["interval_ms"] - 10)
        animation.event_source.interval = state["interval_ms"]
        fig.canvas.draw_idle()
    elif key in {"-", "_"}:
        state["interval_ms"] = min(1000, state["interval_ms"] + 10)
        animation.event_source.interval = state["interval_ms"]
        fig.canvas.draw_idle()
    elif key == "left":
        if not state["paused"]:
            animation.event_source.stop()
            state["paused"] = True
        state["frame"] = (state["frame"] - 1) % total_frames
        _draw_current_frame()
        fig.canvas.draw_idle()
    elif key == "right":
        if not state["paused"]:
            animation.event_source.stop()
            state["paused"] = True
        state["frame"] = (state["frame"] + 1) % total_frames
        _draw_current_frame()
        fig.canvas.draw_idle()
    elif key == "r":
        state["direction"] *= -1
        fig.canvas.draw_idle()
    elif key == "q":
        plt.close(fig)

animation = FuncAnimation(
    fig,
    update,
```

```
        frames=total_frames,
        interval=state["interval_ms"],
        blit=False,
        repeat=True,
    )
    fig.canvas.mpl_connect("key_press_event", on_key)
    _ANIMATION_HOLD.append(animation)

    plt.tight_layout()
    plt.show()
    return animation
```